



ULPGC
Universidad de
Las Palmas de
Gran Canaria



ESCUELA DE
INGENIERÍA INFORMÁTICA

Trabajo de Fin de Grado

Interpretación del lenguaje de signos español basada en visión por computador y aprendizaje profundo

TITULACIÓN: Grado en Ciencia e Ingeniería de Datos

AUTOR: Gerardo León Quintana

TUTORIZADO POR:

José Javier Lorenzo Navarro

José Ignacio Salas Cáceres

Fecha [07/2025]

SOLICITUD DE DEFENSA DE TRABAJO DE FIN DE TÍTULO

D. Gerardo León Quintana, autor del trabajo 'Interpretación del lenguaje de signos español basada en visión por computador y aprendizaje profundo', correspondiente a la titulación Grado en Ciencia e Ingeniería de Datos.

SOLICITA

que se inicie el procedimiento de defensa del mismo, para lo que se adjunta la documentación requerida, haciendo constar que

☒ se autoriza / ☐ no se autoriza la grabación en audio de la exposición y turno de preguntas.

Asimismo, con respecto al registro de la propiedad intelectual/industrial del TFT, declara que:

☐ Se ha iniciado o hay intención de iniciarlo (defensa no pública).

☒ No está previsto.

Y para que así conste firma la presente.

El/La estudiante

LEON
QUINTANA
GERARDO -
45394865H

Firmado digitalmente
por LEON QUINTANA
GERARDO -
45394865H
Fecha: 2025.07.09
10:17:48 +01'00'

Fdo. _____

A rellenar y firmar **obligatoriamente** por el/la/los/las tutores

En relación con la presente solicitud, se informa:

Positivamente (en caso de detección de copia, esta firma quedará invalidada)

LORENZO
NAVARRO
JOSE
JAVIER -
42840825J

Digitally signed
by LORENZO
NAVARRO JOSE
JAVIER -
42840825J
Date:
2025.07.09
20:26:41 +01'00'

SALAS
CACERES
JOSE
IGNACIO -
44741262Y

Firmado
digitalmente
por SALAS
CACERES JOSE
IGNACIO -
44741262Y
Fecha:
2025.07.09
20:48:35
+01'00'

Fdo. _____

Negativamente (justificación en TFT05)

Fdo. _____

DIRECTOR DE LA ESCUELA DE INGENIERÍA INFORMÁTICA

Agradecimientos

Quiero comenzar expresando mi más profundo agradecimiento a mi familia, por haber sido un apoyo incondicional a lo largo de toda mi formación, por celebrar mis logros y consolarme en los momentos difíciles. En especial, deseo hacer una mención honorífica a mis abuelos, Fernando José León y Francisco Quintana. Ambos han sido referentes fundamentales en mi desarrollo personal, de ellos aprendí a enfrentar cada desafío con respeto, a dar siempre lo mejor de mí y a procurar, ante todo, el bienestar de mi familia

También quiero agradecer a todas las amistades y personas cercanas que me han acompañado durante esta etapa universitaria: en trabajos en grupo, mañanas y tardes de estudio, y alguna que otra salida. En especial, a mi pareja, por acompañarme en los momentos de mayor estrés a lo largo de la carrera, por motivarme a seguir mis metas y por animarme siempre a dar lo mejor de mí.

Por supuesto, no puedo dejar de mencionar a alguien que, aunque no ha podido acompañarme en estos últimos años, ha sido a lo largo de mi vida un rival, un cómplice y, sobre todo, un gran amigo. Pablo, ¿qué puedo decirte que no te haya dicho ya? Si hoy estoy aquí, es en gran parte gracias a ti. No me imagino haber recorrido este camino sin nuestros 'piques' en los exámenes y sin ese apoyo mutuo que siempre nos ha impulsado a mejorar.

Por último, pero no por ello menos importante, quiero agradecer al doctor José Javier Lorenzo Navarro y a José Ignacio Salas Cáceres el honor de haberme aceptado como alumno. Ha sido un auténtico orgullo trabajar junto a ustedes durante estos cinco meses. No tengo palabras suficientes para expresar mi gratitud por su paciencia, el conocimiento que me han transmitido y el apoyo que siempre me han brindado.

Resumen

Este trabajo de fin de grado propone nuevos métodos para la interpretación automática de la Lengua de Signos Española, con el objetivo de fomentar la inclusión social de las personas sordas mediante el desarrollo de tecnologías accesibles. Para ello, se emplean técnicas de visión por computador y aprendizaje profundo. A partir del conjunto de datos LSE-Health-UVigo, se han procesado vídeos de intérpretes mediante herramientas como MediaPipe y OpenCV, extrayendo coordenadas articulares del cuerpo, rostro y manos. Sobre estos datos, se entrenaron modelos basados en redes neuronales recurrentes, como LSTM y GRU, orientados a tareas de clasificación de gestos aislados, reconocimiento de gestos continuos y detección de novedades.

Además, se exploraron técnicas de aprendizaje métrico como *Triplet Loss* para construir espacios de características más discriminativos, y se implementaron arquitecturas basadas en *Temporal Convolutional Networks* para realizar segmentación automática de gestos dentro de secuencias no segmentadas.

Los resultados experimentales demuestran la viabilidad de combinar diferentes enfoques del aprendizaje profundo para mejorar el reconocimiento automático de gestos en lengua de signos, incluso en escenarios abiertos y no controlados. Este trabajo supone una contribución tanto técnica como social, proponiendo un sistema que puede facilitar la comunicación entre personas sordas y oyentes en entornos educativos, laborales y sanitarios.

Abstract

This bachelor’s thesis proposes novel methods for the automatic interpretation of Spanish Sign Language, aiming to promote the social inclusion of deaf individuals through the development of accessible technologies. To achieve this, the project applies computer vision and deep learning techniques. Using the LSE-Health-UVigo dataset, videos of sign language interpreters were processed with tools such as MediaPipe and OpenCV to extract joint coordinates from the body, face, and hands. Based on these features, recurrent neural network models, including LSTM and GRU, were trained to perform tasks such as isolated gesture classification, continuous gesture recognition, and novelty detection.

Additionally, metric learning techniques such as *Triplet Loss* were explored to build more discriminative feature spaces, and architectures based on *Temporal Convolutional Networks* were implemented to enable automatic segmentation of gestures within unsegmented video sequences.

Experimental results demonstrate the feasibility of combining various deep learning approaches to improve automatic sign language gesture recognition, even under open and uncontrolled conditions. This work contributes both technically and socially by proposing a system that can facilitate communication between deaf and hearing individuals in educational, workplace, and healthcare settings.

Índice general

1. Introducción	1
2. Competencias y Objetivos de desarrollo sostenible	5
2.1. Competencias específicas	5
2.2. Alineamiento con los objetivos de desarrollo sostenible	6
3. Estado actual	8
3.1. Lengua de signos	8
3.2. Visión por computador para el reconocimiento de la lengua de signos	8
3.3. Métodos y Herramientas	12
3.3.1. Herramientas de Visión por Computador	12
3.3.2. Aprendizaje profundo y modelado secuencial	13
3.3.3. Detección de novedades	20
3.3.4. Triplet Loss y aprendizaje métrico	22
3.3.5. Segmentación temporal de gestos	23
3.3.6. Métricas de evaluación	29
4. Desarrollo	31
4.1. Metodología	31
4.2. Planteamiento del Proyecto	32
4.3. Comprensión de los Datos	34
4.4. Procesamiento de los Datos	38
4.5. Modelado	40
4.5.1. Modelado para un clasificador de gestos aislados	40
4.5.2. Detección de gestos continuos en videos	47
4.5.3. Detección de novedades en gestos	48
4.5.4. Entrenamiento por tripletas para el reconocimiento de gestos	52
4.5.5. Segmentación de gestos en videos	54
5. Evaluación	58
5.1. Clasificación de gestos aislados	58
5.2. Reconocimiento de gestos en video	59
5.3. Detección de novedades	59
5.4. Representaciones por tripletas	59

5.5. Segmentación automática de gestos	59
5.6. Evaluación completa del proyecto	60
6. Conclusiones y trabajo futuro	62
6.1. Conclusiones	62
6.2. Trabajo futuro	63

Índice de Algoritmos

1.	Actualización de parámetros con AdaGrad	41
2.	Actualización de parámetros con RMSProp	42
3.	Actualización de parámetros con Adam	43
4.	Submuestreo por Cajas (Box-based Downsampling)	56

Índice de figuras

3.1. Ejemplos de sensores empleados para el registro de lengua de signos	12
3.2. Detección de puntos de referencia en la mano	13
3.3. Detección de puntos de referencia del cuerpo	14
3.4. Representación de la arquitectura RNN	15
3.5. Representación de una celda de la arquitectura LSTM	16
3.6. Representación de una celda de la arquitectura GRU	18
3.7. Representación del hiperplano de un SVM	21
3.8. Representación del método Triplet Loss	22
3.9. Representación de la arquitectura ED-TCN	26
3.10. Representación de la arquitectura D-TCN	27
4.1. Ciclo de vida de CRISP-DM	33
4.2. Muestra del conjunto de datos	34
4.3. Ejemplo del intérprete eliminado por adoptar una postura no natural	35
4.4. Representación del número de muestras por clase de gesto	36
4.5. Gráfica de las duraciones de vídeos de gestos aislados	37
4.6. Representación de la curva ROC de OC-SVM	51

Índice de Tablas

2.1. Grado de relación del TFT con los objetivos de desarrollo sostenible.	7
4.1. Representación Bag of Words de tres frases	36
4.2. Evaluación de modelos con el conjunto de relleno	46
4.3. Evaluación de modelos con el conjunto de muestreo.	47
4.4. Rendimiento del modelo OC-SVM	50
4.5. Comparación de desempeño entre estrategias de Triplet Loss	53

Capítulo 1

Introducción

La inclusión de personas con discapacidad auditiva ha avanzado considerablemente en ámbitos como la educación, las instituciones y la vida social. No obstante, persisten desafíos relevantes para garantizar una integración plena, especialmente en lo que respecta al acceso equitativo a la información, la comunicación fluida y la participación activa en todos los contextos.

En particular, la comunidad sorda, compuesta por más de 70 millones de personas en el mundo según la Federación Mundial de Sordos (WFD) [1], con más del 80 % viviendo en países en desarrollo, siguen luchando contra las barreras de comunicación en su vida cotidiana. Entre ellas destacan la escasez de intérpretes de lengua de signos y las dificultades de comunicación de cara a servicios esenciales como la educación, la sanidad o la administración pública.

Actualmente, la interpretación en lengua de signos depende casi exclusivamente de profesionales humanos. Sin embargo, el número limitado de intérpretes cualificados no permite cubrir la creciente demanda, dificultando su disponibilidad en situaciones cotidianas o de urgencia.

En el plano tecnológico, la interacción hombre-máquina (HMI) ha evolucionado notablemente en los últimos años, incorporando tecnologías capaces de interpretar señales acústicas y visuales con alta precisión. Estos avances se han aplicado sobre todo al reconocimiento de voz, la identificación biométrica, el análisis de emociones y la traducción automática. Sin embargo, el reconocimiento automático de la lengua de signos sigue siendo un área menos desarrollada, debido a su complejidad inherente, al tratarse de un lenguaje que combina simultáneamente movimientos de manos, expresiones faciales, posturas corporales y relaciones espaciales, a diferencia del lenguaje oral.

En este contexto, la visión por computador y el aprendizaje profundo han empezado a mostrar un gran potencial para abordar estos desafíos. Estas tecnologías permiten analizar secuencias de vídeo y extraer patrones gestuales con gran precisión. No obstante, aún existen múltiples retos técnicos, como la variabilidad entre intérpretes, la escasez de datos anotados, la segmentación de gestos en vídeos continuos o la detección de signos desconocidos en entornos abiertos.

En este contexto, el presente trabajo de fin de grado surge con la motivación de contribuir a la creación de soluciones tecnológicas más inclusivas, explorando el potencial que ofrece la visión por computador para la interpretación automática de la lengua de signos española. El objetivo es investigar cómo los métodos actuales, especialmente aquellos basados en visión por computador y aprendizaje profundo, pueden aplicarse en este dominio para el desarrollo de sistemas más accesibles y eficaces. Esta línea de investigación no solo representa un desafío técnico apasionante desde un punto de vista académico, sino también una oportunidad de generar un impacto social real al reducir las barreras de comunicación entre las personas sordas y oyentes. Así, este proyecto se plantea como una contribución tecnológica con una fuerte vocación humanitaria, orientada hacia la ruptura de barreras comunicativas y la inclusividad de la comunidad sorda en la sociedad.

Dicho objetivo puede ser desglosado en los siguientes subobjetivos a modo de etapas para conseguir alcanzar el objetivo principal:

- ✓ Realizar una revisión exhaustiva sobre documentaciones de la lengua de signos mediante visión por computador.
- ✓ Seleccionar y preparar un conjunto de datos adecuado para el entrenamiento y evaluación del sistema.
- ✓ Implementar y entrenar modelos de aprendizaje profundo para el reconocimiento de gestos.
- ✓ Evaluar el rendimiento del sistema desarrollado mediante métricas estándar.
- ✓ Analizar los resultados obtenidos y proponer mejoras o futuras líneas de investigación.

En lo que respecta a las posibles aportaciones de este trabajo de fin de grado, se considera que está estrechamente ligado al ámbito social, ya que se trata de un sistema orientado a promover la inclusión y accesibilidad del lenguaje de signos en la vida cotidiana mediante técnicas de visión por computador y aprendizaje profundo.

Con el método desarrollado, se plantea facilitar la comunicación entre personas usuarias de la lengua de signos, especialmente aquellas que dependen exclusivamente de ella para comunicarse, y personas que desconocen dicha lengua. Además de mejorar la interacciones humano-máquina de este tipo de usuarios.

Tal como se ha mencionado, una herramienta con estas características tendría un impacto significativo no solo en el ámbito social, sino también en el contexto laboral, al favorecer la inserción y participación activa de personas sordas, mudas o sordomudas en entornos laborales diversos, mejorando así su interacción con clientes, compañeros y superiores. Asimismo, este sistema podría tener un uso particularmente valioso en el ámbito educativo, al facilitar la comunicación entre alumnos y profesores, y promover así una educación más inclusiva.

Finalmente, se destaca que, más allá de las implicaciones prácticas mencionadas, este proyecto contribuye a nivel técnico o científico al proponer e implementar una metodología novedosa para la detección y reconocimiento automático de signos específicos de la lengua de signos española, abriendo nuevas vías para futuras investigaciones e innovaciones tecnológicas orientadas a la accesibilidad y la inclusión social.

En línea con lo anterior, se ha optado por el uso de modelos de aprendizaje profundo (*Deep Learning*). Dentro del amplio espectro de arquitecturas disponibles en este campo, se ha seleccionado una familia de modelos especialmente diseñados para el tratamiento de datos secuenciales: las redes neuronales recurrentes (*Recurrent Neural Networks*, RNN) [2]. Basándose en dichas arquitecturas, se han implementado dos variantes populares y eficaces: las *Gated Recurrent Units* (GRU) [3] y las *Long Short-Term Memory* (LSTM) [4], con el objetivo de abordar tres tareas principales:

- ✓ La **clasificación de gestos aislados**, donde se entrena un modelo para reconocer de forma supervisada el significado de un gesto específico.
- ✓ La **detección de novedad** (*novelty detection*), que consiste en identificar cuándo un gesto no pertenece a ninguna de las clases conocidas previamente por el modelo.
- ✓ Y la detección de gestos en vídeos continuos, es decir, localizar su posición temporal dentro de secuencias no segmentadas.

El conjunto de gestos utilizado ha sido extraído a partir del corpus LSE-Health-UVigo [5], el cual ha sido cedido por la Universidad de Vigo para la realización de este proyecto. Dicho corpus contiene vídeos de diferentes intérpretes de la Lengua de Signos Española explicando conceptos relacionados con el ámbito de la salud, incluyendo enfermedades cardiovasculares, respiratorias, digestivas, entre otras.

A partir de dichos vídeos, se ha empleado la herramienta MediaPipe [6], desarrollada por Google, para extraer automáticamente los puntos de interés (*landmarks*) de los intérpretes, incluyendo coordenadas de manos, rostro y cuerpo. Estos puntos representan una codificación estructurada del movimiento gestual, lo que permite transformar los vídeos en secuencias numéricas aptas para el entrenamiento de modelos.

Sobre estas representaciones se han aplicado distintos enfoques experimentales. Por un lado, se ha analizado el rendimiento de los modelos GRU y LSTM utilizando técnicas de *padding* y muestreo para la normalización temporal de los datos.

Por otro, se ha estudiado la eficacia de funciones de pérdida alternativas, como la *Focal Loss*, y se han evaluado estrategias de aprendizaje métrico mediante *Triplet Loss*, para mejorar la discriminación entre gestos visualmente similares.

Además, se ha aplicado un modelo de segmentación basado en *Temporal Convolutional Networks* (**ED-TCN**) para detectar automáticamente los intervalos temporales donde se producen los gestos dentro de los vídeos continuos.

Finalmente, se han llevado a cabo experimentos de detección de novedades mediante técnicas como **One-Class SVM** y análisis de distancias en el espacio de vectores de características generado por los modelos, con el fin de dotar al sistema de capacidad para operar en entornos abiertos donde pueden aparecer gestos desconocidos.

El resto del documento se organiza de la siguiente forma:

- ✓ En el **Capítulo 2** se presenta el estado del arte, donde se comenta el estado actual de la lengua de signos, las técnicas de visión por computador orientadas a la lengua de

signos, las arquitecturas implementadas y herramientas de visión por computador.

- ✓ En el **Capítulo 3** se detallan las aportaciones y competencias desarrolladas en el proyecto, así como su alineación con los Objetivos de Desarrollo Sostenible y las competencias del grado.
- ✓ El **Capítulo 4** describe en profundidad el desarrollo del sistema, el preprocesamiento de datos, la implementación de modelos y los procesos de entrenamiento y validación.
- ✓ En el **Capítulo 5** se realiza una breve recapitulación de los resultados obtenidos en las fases de modelado, y posteriormente una evaluación completa del proyecto.
- ✓ Finalmente, el **Capítulo 6** presenta las conclusiones y trabajo futuro, donde se valoran los logros alcanzados, se discuten las limitaciones del enfoque actual y se proponen posibles líneas de investigación para continuar este trabajo.

Capítulo 2

Competencias y Objetivos de desarrollo sostenible

En este capítulo se mencionarán las competencias específicas relacionadas con este y finalmente se explicarán los objetivos de desarrollo sostenible con los que cumple el trabajo.

2.1. Competencias específicas

Adentrándonos en las competencias específicas aplicadas al desarrollo de este trabajo de fin de grado, serán desglosadas a continuación:

ED1

Capacidad para conocer los fundamentos, paradigmas y técnicas propias de los sistemas inteligentes y analizar, diseñar y construir sistemas, servicios y aplicaciones informáticas que utilicen dichas técnicas en cualquier ámbito de aplicación.

Esta competencia se ha abordado mediante la selección, diseño y puesta en práctica de modelos de aprendizaje profundo para la detección de gestos. Se han implementado redes neuronales recurrentes (LSTM, GRU) para extraer patrones temporales de datos provenientes de sistemas inteligentes basados en visión artificial (MediaPipe). Este enfoque permitió construir un modelo clasificador capaz de clasificar gestos aislados de múltiples intérpretes.

ED2

Capacidad para adquirir, obtener, formalizar y representar el conocimiento humano en una forma computable para la resolución de problemas mediante un sistema informático en cualquier ámbito de aplicación, particularmente los

relacionados con aspectos de computación, percepción y actuación en ambientes o entornos inteligentes.

La adquisición, formalización y representación del conocimiento se llevó a cabo mediante la creación de vectores de características representativos extraídos a partir de los videos y de las transcripciones de dichos videos. Estos vectores de características constituyen una forma computable del propio conocimiento humano, obteniendo en sí las características distintivas que posee cada gesto realizado por los intérpretes.

ED4

Capacidad de identificar y analizar problemas y diseñar, desarrollar, implementar, verificar y documentar soluciones software en ámbitos de aplicación de la Inteligencia Artificial en Ciencia e Ingeniería de Datos.

Se identificaron claramente los desafíos inherentes a la detección automática de gestos en videos, realizando un análisis exhaustivo del problema desde la extracción y preprocesamiento de datos hasta el desarrollo, implementación y validación de soluciones basadas en inteligencia artificial.

En definitiva, este trabajo ha permitido aplicar y consolidar estas competencias específicas mediante la propuesta de resolución de un problema real, combinando técnicas avanzadas de inteligencia artificial con un enfoque práctico orientado a mejorar la interpretación de la lengua de signos.

2.2. Alineamiento con los objetivos de desarrollo sostenible

En función de la relación indicada en la Tabla 2.1, se justifica a continuación la alineación de este trabajo fin de grado con los Objetivos de Desarrollo Sostenible (ODS) identificados:

- ✓ **ODS 4 - Educación de calidad:** El proyecto tiene implicaciones, en el ámbito educativo, ya que propone una metodología, mediante la inteligencia artificial, que podría agilizar la comunicación y la participación activa en procesos educativos por parte de personas que requieran de la lengua de signos como único método de comunicación. Al implementar técnicas avanzadas de inteligencia artificial aplicadas a la interpretación de la lengua de signos, se contribuye a promover entornos educativos más inclusivos y accesibles.
- ✓ **ODS 10 - Reducción de las desigualdades:** Este trabajo tiene una clara y directa relación con la reducción de desigualdades, especialmente las que enfrentan personas con discapacidad auditiva o del habla. Mediante el desarrollo de sistemas automáticos de interpretación de la lengua de signos, se potencia la inclusión social, laboral y educativa de estas personas. Al facilitar su comunicación con el resto de la sociedad, se contribuye

de forma concreta y significativa a la reducción de barreras comunicativas, impulsando así una sociedad con una mayor equidad social e inclusividad.

En consecuencia, el presente trabajo se alinea de manera específica con estos ODS, promoviendo un impacto social positivo mediante la aplicación innovadora de tecnologías avanzadas en pro de la inclusión y la accesibilidad.

Cuadro 2.1: Grado de relación del TFT con los objetivos de desarrollo sostenible.

ODS	Grado de relación			
	0 No procede	1 Bajo	2 Medio	3 Alto
1 Fin de la Pobreza	X			
2 Hambre cero	X			
3 Salud y Bienestar	X			
4 Educación de calidad			X	
5 Igualdad de género	X			
6 Agua limpia y saneamiento	X			
7 Energía Asequible y no contaminante	X			
8 Trabajo decente y crecimiento económico	X			
9 Industria, Innovación e Infraestructuras	X			
10 Reducción de las desigualdades				X
11 Ciudades y comunidades sostenibles	X			
12 Producción y consumo sostenibles	X			
13 Acción por el clima	X			
14 Vida submarina	X			
15 Vida de ecosistemas terrestres	X			
16 Paz, justicia e instituciones sólidas	X			
17 Alianzas para lograr objetivos	X			

Capítulo 3

Estado actual

En esta sección se hablará sobre el estado del arte de la lengua de signos, la visión por computador y diversos aspectos del aprendizaje profundo.

3.1. Lengua de signos

La lengua de signos es una lengua visual y gestual, usada por las comunidades sordomudas para comunicarse. A diferencia de las lenguas orales, se basa en movimientos manuales, expresiones faciales, posiciones del cuerpo y uso del espacio tridimensional. Cada país posee su propia lengua de signos, y cada una cuenta con su propia gramática y semántica.

La lengua de signos no se considera un lenguaje como tal, citando la definición del Diccionario de la Real Academia Española (DRAE) de las palabras 'lengua' y 'lenguaje', la lengua se define como 'Sistema lingüístico considerado en su estructura', mientras que el lenguaje se define como la 'Facultad del ser humano de expresarse y comunicarse con los demás a través del sonido articulado o de otros sistemas de signos'. Por tanto, como la lengua de signos no utiliza el sonido articulado sino un sistema propio de signos visuales y gestuales, encaja más adecuadamente dentro de la definición de 'lengua' que dentro de la de 'lenguaje', en el sentido en que el DRAE reserva este último término para la capacidad general de comunicación. Así, pese a ciertas confusiones terminológicas, la lengua de signos cumple con las características de un sistema lingüístico completo, con reglas gramaticales, léxico y sintaxis propios, y debe ser reconocida como una lengua en pleno derecho, al igual que las lenguas orales.

3.2. Visión por computador para el reconocimiento de la lengua de signos

El reconocimiento automático de lengua de signos (Sign Language Recognition, SLR) mediante el uso de la visión por computador, constituye a una disciplina clave dentro del

ámbito de la interacción hombre-máquina. Esta rama de la visión por computador resulta especialmente relevante para personas sordas o con dificultades del habla, permitiendo la interpretación de signos mediante sistemas automáticos. En la literatura especializada, se distinguen principalmente dos enfoques: el reconocimiento de signos aislados (Isolated Sign Language Recognition, ISLR) y el reconocimiento continuo (Continuous Sign Language Recognition, CSLR), cada uno con desafíos y características técnicas particulares.

Reconocimiento de signos aislados (ISLR)

El reconocimiento de signos aislados representa una de las áreas más consolidadas dentro del reconocimiento automático de lengua de signos. Su objetivo es identificar gestos individuales de forma segmentada, donde los límites temporales de cada signo están claramente definidos. Esta característica lo convierte en un enfoque especialmente adecuado para tareas de clasificación supervisada, ya que cada muestra de vídeo corresponde inequívocamente a una única glosa, las glosas son las transcripciones de las señas o gestos a un medio escrito representando así su concepto. No obstante, su aparente simplicidad oculta desafíos técnicos y metodológicos significativos.

En cuanto a las modalidades de entrada, la mayoría de los trabajos actuales emplean secuencias de vídeo RGB como principal fuente de información visual [7, 8, 9]. Esta modalidad ofrece una representación rica de manos, rostro y postura corporal, aunque sufre limitaciones en presencia de condiciones adversas como variaciones de iluminación, oclusiones parciales o fondos complejos [10].

Para paliar estas limitaciones, se ha extendido el uso de fuentes de datos complementarias como mapas de profundidad y datos esqueléticos. Mientras que los mapas de profundidad aportan robustez frente a cambios de iluminación, los datos esqueléticos extraídos mediante herramientas como OpenPose [11] o MediaPipe [6] permiten representar de forma compacta y eficiente las poses y trayectorias articulares, lo cual resulta especialmente valioso en entornos con restricciones computacionales [12].

Desde una perspectiva lingüística, los enfoques más efectivos en ISLR modelan tanto características manuales como no manuales [7, 13]. Las primeras incluyen la configuración y orientación de las manos, su ubicación espacial y los perfiles de movimiento; las segundas abarcan expresiones faciales, orientación de la cabeza, movimientos oculares y postura corporal, todos ellos elementos esenciales para transmitir matices semánticos y gramaticales en lengua de signos [14].

En las etapas iniciales del campo, estas características se extraían manualmente mediante técnicas tradicionales de procesamiento de imágenes. Sin embargo, con la adopción del aprendizaje profundo, se han impuesto representaciones aprendidas automáticamente a partir de fotogramas completos o regiones específicas, frecuentemente potenciadas por mecanismos de atención [15].

Un aspecto metodológico central en ISLR es la integración de múltiples modalidades de entrada. Para ello, se han desarrollado distintas estrategias de fusión que se definen en el artículo [7] como *Early Fusion* y *Late Fusion*.

La estrategia de fusión *Early Fusion* hace referencia a uno de los distintos niveles de fusión que se mencionan en el artículo [16], concretamente a la fusión a nivel de características. Como su nombre indica esta estrategia de fusión se centra en combinar o fusionar los distintos canales de entrada de información del modelo con el objetivo de generar una muestra única.

Por otro lado, la estrategia de fusión *Late Fusion* corresponde con la fusión a nivel de puntuación [16], dicho nivel de fusión se basa en el uso de diferentes comparadores y posteriormente fusionar las puntuaciones obtenidas a través de dichos comparadores para entrenar al algoritmo.

La fusión a nivel de puntuación ha destacado por encima de la estrategia de fusión a nivel de características debido a su simplicidad arquitectónica, y su capacidad para combinar eficazmente predicciones provenientes de diferentes canales, como vídeo RGB y datos esqueléticos, sin requerir una estricta sincronización temporal [17].

Finalmente, la escasez de grandes conjuntos de datos etiquetados en lengua de signos ha impulsado el uso del aprendizaje por transferencia (*transfer learning*) como estrategia fundamental en ISLR. Inicialmente, se recurrió a modelos preentrenados en conjuntos de imágenes como ImageNet [18], aprovechando sus representaciones visuales generales. En los últimos años, se han empleado redes preentrenadas en tareas de reconocimiento de acciones humanas, como I3D sobre el conjunto Kinetics [19], lo que ha mejorado sustancialmente la captura de información temporal. Asimismo, comienzan a emerger enfoques de transferencia intra-dominio, en los que un modelo se entrena en un corpus de lengua de signos de gran escala y se adapta posteriormente a dominios más específicos, lo que contribuye a mitigar el sobreajuste y mejora la capacidad de generalización [7].

Reconocimiento continuo de lengua de signos (CSLR)

Por otro lado, el CSLR aborda el reconocimiento de secuencias de signos ejecutadas de manera fluida, sin pausas delimitadas entre cada gesto. De acuerdo con Alyami et al. [20], este tipo de reconocimiento se plantea como un problema de secuencia a secuencia, donde se busca mapear un conjunto de fotogramas $X = \{x_1, x_2, \dots, x_n\}$ a una secuencia de glosas $Y = \{y_1, y_2, \dots, y_m\}$, siendo generalmente $n \neq m$ en general. El sistema debe identificar los límites temporales de cada signo y reconocer su significado con precisión.

Los sistemas modernos de CSLR suelen organizarse en tres módulos fundamentales:

- ✓ **Módulo Espacial:** en este módulo se extraen las características visuales del fotograma al completo.
- ✓ **Módulo Temporal:** se encarga de modelar la dinámica de los gestos realizados por los intérpretes.
- ✓ **Módulo de Alineación:** es el módulo encargado de vincular los datos de entrada con las etiquetas de salida.

Para estos fines, se utilizan arquitecturas como redes convolucionales 3D (C3D), redes

recurrentes (RNN, LSTM), mecanismos de atención y técnicas de conexión temporal (*Connectionist Temporal Classification*).

Entre los principales desafíos que enfrenta el reconocimiento continuo de lengua de signos se encuentran una serie de dificultades tanto computacionales como lingüísticas que lo distinguen notablemente de otras variantes del reconocimiento, como el reconocimiento de signos aislados o deletreo manual. En particular, Alyami et al. [20] identifican cinco problemáticas esenciales que limitan la robustez y generalización de los sistemas actuales de CSLR:

1. Segmentación temporal y detección de límites entre signos. Uno de los retos fundamentales consiste en identificar con precisión los puntos de inicio y finalización de cada signo dentro de una secuencia continua. A diferencia del ISLR, donde las señales están previamente segmentadas, el CSLR opera sobre flujos ininterrumpidos de vídeo, donde no existen pausas explícitas que separen los signos. Este hecho dificulta el proceso de reconocimiento, ya que errores en la segmentación afectan directamente a la clasificación posterior. Si bien se han propuesto enfoques explícitos de segmentación [21, 22], estos presentan limitaciones en cuanto a su precisión. Por esta razón, técnicas basadas en modelos ocultos de Markov (HMM) y en *Connectionist Temporal Classification* han sido adoptadas para realizar una alineación automática de las secuencias sin segmentación previa.

2. Movimientos de epéntesis (*movement epenthesis*). En el lenguaje de signos, es frecuente que se produzcan movimientos transitorios entre signos consecutivos que no corresponden a ninguna glosa particular. Estos movimientos no lingüísticos, conocidos como epéntesis, introducen ruido visual y semántico en las secuencias, lo que complica la tarea de los modelos de CSLR. Algunos enfoques han tratado explícitamente su detección y eliminación [23, 24], mientras que otros proponen abordajes implícitos que integran la variabilidad dentro del propio modelo [25, 26].

3. Coarticulación entre signos. Un fenómeno inherente a las lenguas signadas es la coarticulación, es decir, la modificación de la forma de un signo debido al contexto gestual inmediato, como pueden ser los signos que lo preceden o que lo prosiguen. Esto incrementa la variabilidad intra clase y puede generar ambigüedad al momento de reconocer gestos visualmente similares [27]. Para afrontarlo, se requiere que los modelos capten relaciones temporales finas que consideren el contexto gestual completo.

4. Dependencias entre signos. A diferencia del ISLR, donde los gestos se reconocen de forma independiente, el CSLR requiere modelar relaciones de dependencia a largo plazo entre los signos de una secuencia. En este sentido, algunos modelos tradicionales, como los HMM, asumen independencia entre signos y concatenan predicciones individuales, lo que compromete la coherencia contextual. En cambio, enfoques más recientes emplean modelos secuenciales capaces de capturar información semántica de alto nivel, como las redes LSTM o arquitecturas basadas en Transformers [20].

5. Inclusión de signos deletreados. En conversaciones reales en lengua de signos, es común que se incorporen elementos deletreados, por ejemplo, nombres propios o palabras técnicas, que requieren la representación de letras individuales. Estos signos tienden a ser breves y poco dinámicos, lo que complica su diferenciación respecto a los movimientos epentéticos. Además, pueden solaparse con signos dinámicos que también emplean configu-

raciones dactilares similares. Desarrollar modelos que integren el reconocimiento conjunto de signos dinámicos y deletreados sigue siendo una línea de investigación activa y desafiante.

En conjunto, estos desafíos resaltan la complejidad inherente al CSLR y la necesidad de arquitecturas sofisticadas que integren mecanismos de segmentación, representación contextual, robustez multimodal y adaptación al estilo del intérprete. Superar estas barreras es esencial para avanzar hacia sistemas de interpretación automática de signos más precisos y aplicables en entornos reales.

En cuanto a los datos de entrada, la mayoría de los estudios actuales, aproximadamente el 82 % [20], utilizan vídeos RGB. Aunque los sensores basados en guantes o brazaletes (*armbands*), Figura 3.1 ,proporcionan datos precisos, su alto coste y falta de naturalidad han limitado su adopción en entornos reales. Como alternativa, se ha fomentado el uso de datos multimodales que combinan imágenes RGB, mapas de profundidad y esqueletos, obtenidos a través de dispositivos como Microsoft Kinect, para mejorar la capacidad de generalización de los modelos [20].



Ilustración 3.1: Ejemplos de sensores empleados para el registro de lengua de signos. a) CyberGlove [28], b) Myo armband [29]

Ambos enfoques, ISLR y CSLR, desempeñan un papel crucial en el desarrollo de tecnologías inclusivas basadas en visión por computador. Mientras que ISLR ha alcanzado altos niveles de precisión en contextos controlados, CSLR representa el reto más próximo a una interpretación natural y fluida del lenguaje de signos. La tendencia actual se orienta hacia el uso de arquitecturas híbridas y multimodales que integran ambas estrategias, con el objetivo de lograr modelos robustos, eficientes y capaces de operar en tiempo real en escenarios no controlados.

3.3. Métodos y Herramientas

3.3.1. Herramientas de Visión por Computador

Para la extracción de características se han empleado herramientas de visión por computador como pueden ser OpenCV [30] y MediaPipe [6].

OpenCV es una biblioteca de software de código abierto orientada a la visión por computador y aprendizaje automático. Está diseñada para proporcionar herramientas que permi-

tan desarrollar aplicaciones en tiempo real relacionadas con el procesamiento de imágenes y vídeos. En este trabajo se ha empleado principalmente por sus capacidades para el tratamiento y manipulación de imágenes y vídeos.

MediaPipe, por su parte, es un *framework* de código abierto desarrollado por Google, orientado al desarrollo de *pipelines* de procesamiento de medios visuales en tiempo real. Está especialmente centrado en la visión por computador, utilizando modelos de aprendizaje automático optimizados.

En este trabajo se han empleado dos soluciones de las que dispone Mediapipe, estas se han combinado con el uso de la librería OpenCV para la extracción de características de los gestos realizados por los intérpretes. Entre las soluciones aplicadas encontramos **Hand Tracking**, para la detección y seguimiento de hasta un total de 21 puntos de referencia de la mano [31], como pueden ser las muñecas, puntas de los dedos y sus falanges. Tal y como se observa en la Figura 3.2, este recurso nos permite percibir las diferentes posiciones que tome la mano, pudiendo así analizar el movimiento que los diferentes traductores realicen.

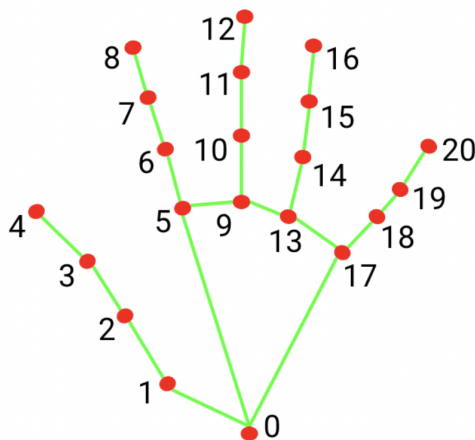


Ilustración 3.2: Detección de puntos de referencia en la mano [31]

Por otro lado, tenemos la segunda solución **Pose Estimation** [32], la cuál es capaz de identificar hasta 33 puntos de referencia del cuerpo humano, lo que nos permitirá conocer los movimientos que involucren a diferentes partes del cuerpo como hombros, codos y cabeza. Toda la información que aporta este modelo se puede observar en la Figura 3.3.

La necesidad de aplicar ambas soluciones radica en la importancia de estos conjuntos de puntos de referencia para la interpretación de la lengua de signos, dado que esta se basa en movimientos articulares y manuales para su comunicación.

3.3.2. Aprendizaje profundo y modelado secuencial

El aprendizaje profundo ha supuesto un punto de inflexión en el ámbito del reconocimiento de patrones, facilitando el entrenamiento de modelos con una capacidad de representación

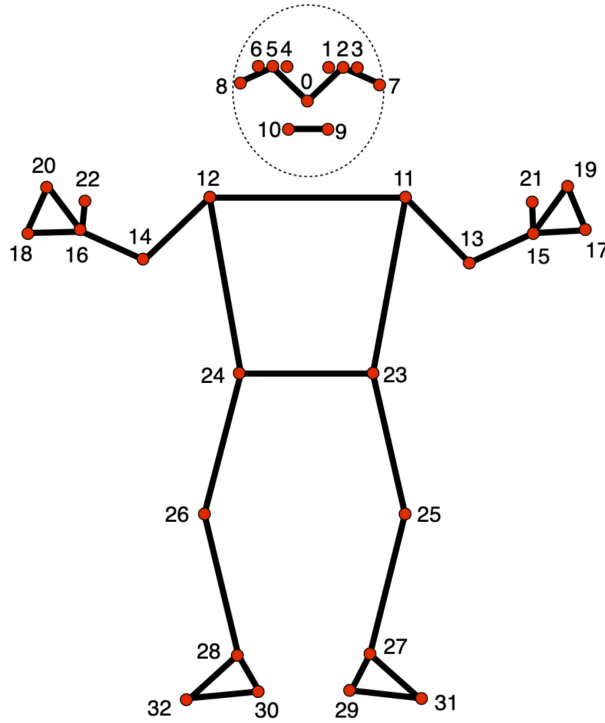


Ilustración 3.3: Detección de puntos de referencia del cuerpo [32]

jerárquica y una elevada cantidad de parámetros. Esta capacidad resulta especialmente relevante en tareas que requieren la interpretación de secuencias temporales, como el reconocimiento de gestos en vídeo.

Redes Neuronales Recurrentes

En este contexto, se han empleado modelos de redes neuronales recurrentes RNN [2], un tipo de arquitectura especialmente adecuada para la modelización de secuencias. A diferencia de las redes neuronales más simples, las RNN están diseñadas para incorporar información temporal, es decir, tienen la capacidad de mantener un estado interno que les permite recordar elementos anteriores en una secuencia. Esta característica las hace útiles para tareas donde el orden y la dependencia temporal entre los datos son fundamentales, como pueden ser el reconocimiento de gestos, el procesamiento de lenguaje natural o el análisis de series temporales.

Generalmente, una RNN procesa una secuencia de entradas $(x_1, x_2, \dots, x_T)$ de forma iterativa, actualizando un estado oculto (h_t) en cada paso temporal (t) , el cual depende tanto de la entrada actual (x_t) como del estado anterior (h_{t-1}) 3.1.

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad (3.1)$$

donde W_{xh} , W_{hh} son los pesos del modelo, b_h es el bias, y f correspondería con la función

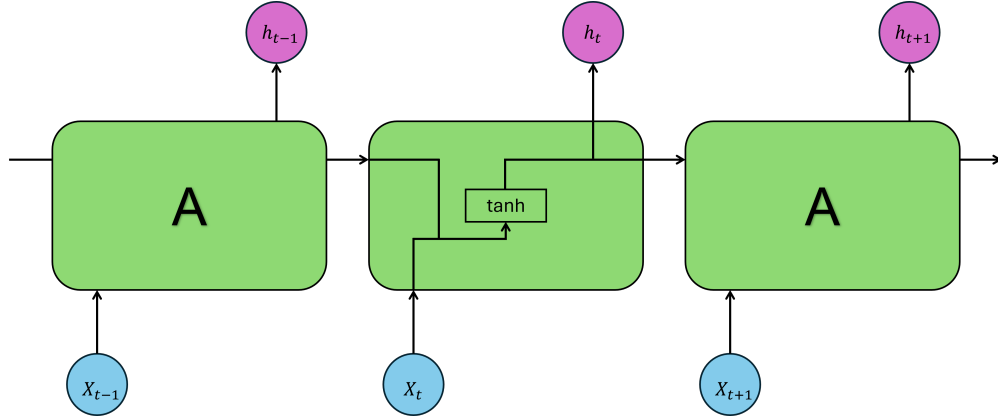


Ilustración 3.4: Representación de la arquitectura RNN

de activación. El estado final o la secuencia completa nos permitirá realizar predicciones o clasificaciones con el modelo.

No obstante, las RNN tradicionales presentan dificultades para aprender dependencias a largo plazo, debido a que presentan el problema del desvanecimiento de gradiente [33].

El problema del desvanecimiento del gradiente ocurre durante la retropropagación de la red, donde el gradiente del error de un estado oculto, por ejemplo $(t - K)$, se calcula como:

$$\frac{\partial E}{\partial h_{t-k}} = \frac{\partial E}{\partial h_t} \cdot \prod_{i=1}^k \frac{\partial h_{t-i+1}}{\partial h_{t-i}} \quad (3.2)$$

Esta expresión describe como se calcula el gradiente del error E con respecto al estado oculto en un instante anterior h_{t-k} , a partir del gradiente en el tiempo actual h_t . Para ello, se encadenan las derivadas parciales de cada estado oculto con respecto al anterior, desde $t - k$ hasta t , lo que equivale a multiplicar sucesivamente los Jacobianos $\frac{\partial h_{t-i+1}}{\partial h_{t-i}}$.

En la práctica, si las normas de estas derivadas son menores que uno, como ocurre habitualmente al utilizar funciones de activación como **tanh** o **sigmoid**, el producto completo decrece exponencialmente con k .

Según el artículo de Bengio et al. [34], aunque las salidas de estas funciones de activación se encuentran en los intervalos $(-1, 1)$ para **tanh** y $(0, 1)$ para **sigmoid**, lo relevante es que las derivadas de ambas funciones también están acotadas por debajo de uno.

En concreto, la derivada de la sigmoide alcanza un máximo de 0,25, mientras que la de **tanh** se mantiene estrictamente menor o igual a 1. Como resultado, al multiplicar sucesivamente estas derivadas durante la retropropagación en el tiempo, el gradiente tiende a decrecer rápidamente, dificultando la actualización efectiva de los pesos y, en consecuencia, el aprendizaje de dependencias a largo plazo.

Long Short Term-Memory

Con el objetivo de evitar este problema, se ha propuesto emplear variantes de la arquitectura RNN como puede ser el caso de las redes Gated Recurrent Unit (GRU) y Long Short Term-Memory (LSTM). Ambas estructuras introducen mecanismos de puertas que permiten regular el flujo de información y facilitar la preservación de gradientes durante la retropropagación, incluso en secuencias largas.

Las redes LSTM, introducidas por Hochreiter y Schmidhuber en 1997 [4], incorporan una memoria interna denominada celda, las cuales pueden conservar información a lo largo de secuencias de mayor tamaño que las RNN tradicionales.

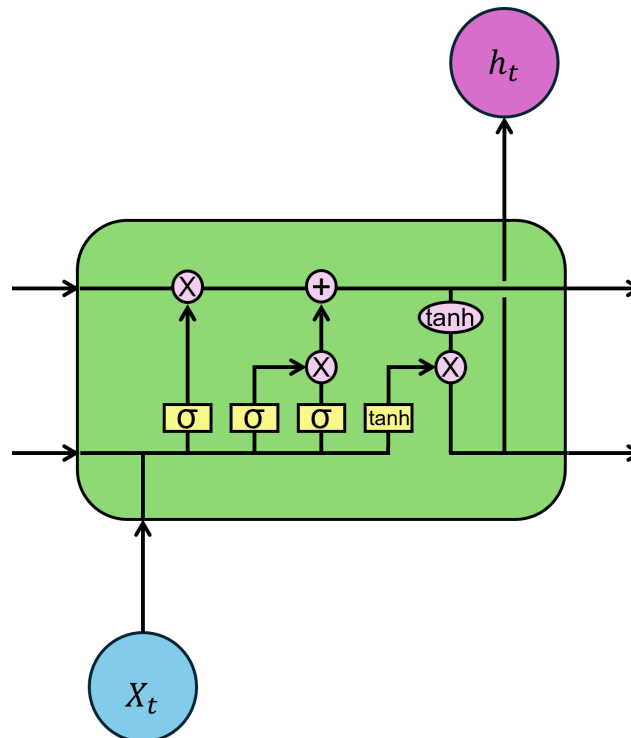


Ilustración 3.5: Representación de una celda de la arquitectura LSTM

Esta celda se gestiona mediante tres mecanismos llamados puertas. Cada puerta se implementa mediante funciones de activación **sigmoide**, que permiten realizar un control continuo sobre la información procesada, y combinaciones con funciones **tanh** para la transformación de estados, lo que nos permite controlar de forma precisa el flujo de información.

En una LSTM tradicional existen tres tipos de puerta:

- ✓ **Puerta de entrada** (*input gate*): determina que parte de la nueva información proveniente de la entrada actual debe almacenarse en la celda de memoria.
- ✓ **Puerta de olvido** (*forget gate*): decide que parte de la información almacenada previamente debe ser eliminada.

- ✓ **Puerta de salida** (*output gate*): regula que información de la celda de memoria se transmite al estado oculto, afectando así a la salida de la red en ese instante.

El funcionamiento de una LSTM puede expresarse con las siguientes ecuaciones:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad (3.3)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad (3.4)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} - 1 + b_c) \quad (3.5)$$

$$c_t = f_t \odot c_{t-1}; +; i_t \odot \tilde{c}_t \quad (3.6)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} - 1 + b_o) \quad (3.7)$$

$$h_t = o_t \odot \tanh(c_t) \quad (3.8)$$

Donde:

- ✓ x_t es la entrada en el instante t .
- ✓ h_{t-1} es el estado oculto en el instante $t - 1$.
- ✓ c_{t-1} es el estado de la celda en el instante $t - 1$.
- ✓ f_t es la salida de la puerta de olvido (*forget gate*).
- ✓ i_t es la salida de la puerta de entrada (*input gate*).
- ✓ $\tilde{c}_t$ es el contenido candidato a almacenar en la celda.
- ✓ c_t es el nuevo estado de la celda.
- ✓ o_t es la salida de la puerta de salida (*output gate*).
- ✓ h_t es el estado oculto (y salida) en el instante t .
- ✓ $W \cdot$ y $U \cdot$ son matrices de pesos para la entrada y el estado oculto, respectivamente.
- ✓ $b \cdot$ son los sesgos asociados a cada puerta.
- ✓ $\odot$ representa el producto de Hadamard (elemento a elemento).
- ✓ σ es la función sigmoide.
- ✓ $\tanh$ es la función tangente hiperbólica.

Gracias a la implementación de estas puertas, las LSTM son capaces de recordar o descartar información de forma selectiva, lo que permite capturar dependencias temporales en un mayor alcance.

Gated Recurrent Unit

Por otro lado, las redes GRU introducidas por Cho et al. en 2014 [3], son una variante de las RNN que buscan evitar el problema del desvanecimiento de gradiente al igual que las LSTM, pero con una arquitectura más simplificada. Las GRU, al igual que las LSTM, emplean el uso de puertas, con la diferencia de que las GRU integran los mecanismos de la puerta de entrada y olvido de la LSTM en un solo estado oculto, sin tener que recurrir a otra celda de memoria separada.

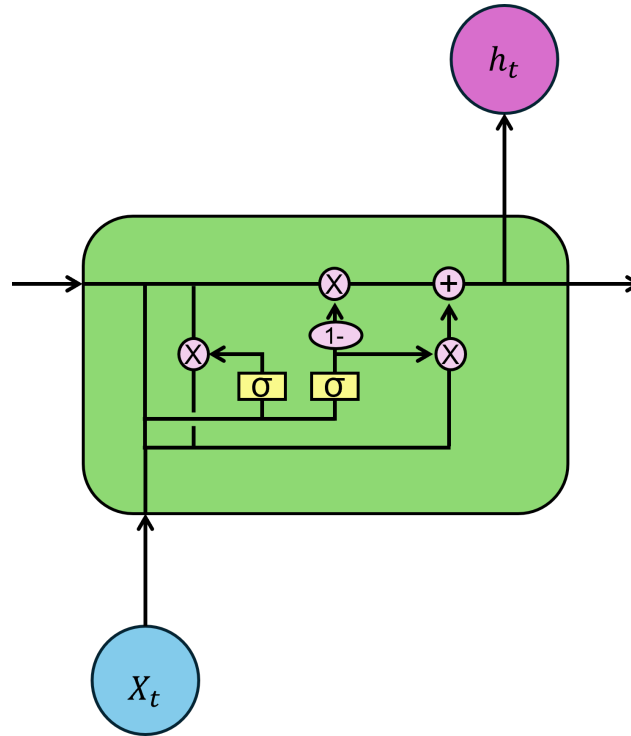


Ilustración 3.6: Representación de una celda de la arquitectura GRU

Esta arquitectura emplea las siguientes puertas:

- ✓ **Puerta de actualización** (*update gate*): controla la cantidad de información del estado anterior que debe conservarse y cuánto del nuevo estado candidato debe incorporarse. Esta puerta unifica el papel de las puertas de entrada y olvido en las LSTM.
- ✓ **Puerta de reinicio** (*reset gate*): determina cuánta información del estado anterior debe ignorarse al generar el nuevo estado candidato.

El funcionamiento de una GRU puede expresarse con las siguientes ecuaciones:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (3.9)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (3.10)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \quad (3.11)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (3.12)$$

Donde:

- ✓ x_t es la entrada en el instante t .
- ✓ h_{t-1} es el estado oculto anterior.
- ✓ $\tilde{h}_t$ es el nuevo estado candidato.
- ✓ z_t y r_t son las salidas de las puertas de actualización y reinicio, respectivamente.
- ✓ $\odot$ representa el producto de Hadamard.
- ✓ σ es la función sigmoide.
- ✓ $\tanh$ es la función tangente hiperbólica.

En base a esta estructura, las GRU son capaces de aprender dependencias temporales tanto a corto como a largo plazo, sin la necesidad de requerir de una memoria explícita como sucede en el caso de las LSTM. Además, al tener un menor número de puertas y de parámetros su entrenamiento posee un menor coste computacional en comparación a las LSTM.

Entre las variantes más destacadas se encuentran las LSTM [4], diseñadas para mitigar los problemas de desvanecimiento y explosión del gradiente característicos de las RNN tradicionales. Estas redes introducen mecanismos de memoria que les permiten aprender dependencias a largo plazo, lo cual ha demostrado ser especialmente útil en tareas como el reconocimiento de acciones humanas, la transcripción automática de lenguaje y la segmentación de secuencias.

Otra arquitectura ampliamente utilizada es la GRU [3], una alternativa más eficiente en términos computacionales. Las GRU reducen la complejidad del modelo al prescindir de ciertos componentes de las LSTM, como la celda de memoria explícita, sin comprometer significativamente el rendimiento. Esto las convierte en una opción preferente en entornos con recursos limitados o cuando se requiere un entrenamiento más rápido.

Ambas arquitecturas han sido empleadas con éxito en múltiples dominios que requieren modelado secuencial, incluyendo la comprensión de lenguaje natural y el análisis de series temporales. En el presente trabajo, se exploran estas redes como mecanismo de modelado temporal sobre secuencias de características derivadas de puntos clave corporales extraídos de vídeos. Su capacidad para capturar la dinámica gestual resulta esencial para abordar de forma eficaz la tarea de interpretación automática del lenguaje de signos.

3.3.3. Detección de novedades

El reconocimiento automático de la lengua de signos enfrenta un reto fundamental: la variabilidad y amplitud del vocabulario gestual en contextos reales. Si bien los modelos actuales logran resultados satisfactorios en conjuntos cerrados y controlados, en entornos abiertos es común que aparezcan gestos que no pertenecen a las clases utilizadas durante el entrenamiento. Este desajuste puede conducir a errores de clasificación en el sistema. Para abordar esta limitación, se propone el uso de técnicas de detección de novedades como posible solución para identificar gestos no observados previamente.

La detección de novedades es una tarea fundamental en entornos abiertos, donde los sistemas pueden enfrentarse a datos que no pertenecen a ninguna de las clases vistas durante el entrenamiento. Este problema resulta especialmente relevante en aplicaciones como el reconocimiento de gestos en lengua de signos, dado que los conjuntos de entrenamiento suelen cubrir únicamente una fracción limitada del vocabulario gestual posible.

Pimentel et al. [35] proponen una clasificación exhaustiva de los métodos de detección de novedades en cuatro grandes categorías: técnicas probabilísticas, enfoques basados en reconstrucción, métodos de distancia y métodos discriminativos. Dentro de esta última categoría, destacan los enfoques de tipo *one-class*, en los cuales se modela exclusivamente la clase positiva sin requerir ejemplos negativos.

Uno de los métodos discriminativos más ampliamente utilizados para detección de novedades es la técnica *One-Class Support Vector Machine* (OC-SVM), introducida por Schölkopf et al. [36]. Esta técnica es una extensión de las máquinas de vectores soporte (*Support Vector Machines*, SVM), un conjunto de métodos de aprendizaje supervisado inicialmente propuesto por Vapnik en 1995 [37], y consolidado por Cortes y Vapnik en su formulación del hiperplano óptimo [38].

Las SVM se utilizan principalmente para clasificación y regresión, y se basan en encontrar un hiperplano que maximice el margen entre clases, minimizando simultáneamente el error de clasificación. Esta idea se ilustra en la Figura 3.7.

En el caso de OC-SVM, el objetivo es determinar una región en el espacio de características que contenga la mayoría de los datos positivos (también llamados *in-distribution*, ID), permitiendo identificar como anomalías o *out-of-distribution* (OOD) aquellas muestras que se sitúan fuera de esta región. A diferencia de las SVM tradicionales, el OC-SVM no requiere ejemplos negativos durante el entrenamiento, lo que lo hace especialmente adecuado para contextos donde la clase de interés es escasa o no etiquetada.

La puntuación asignada por el OC-SVM a una muestra x se define como:

$$s(x) = \mathbf{w}^\top \phi(f(x)) - \rho \quad (3.13)$$

donde $\mathbf{w}$ y ρ son parámetros aprendidos, y ϕ representa la función núcleo utilizada para mapear los datos a un espacio de características de mayor dimensión. Una de las funciones

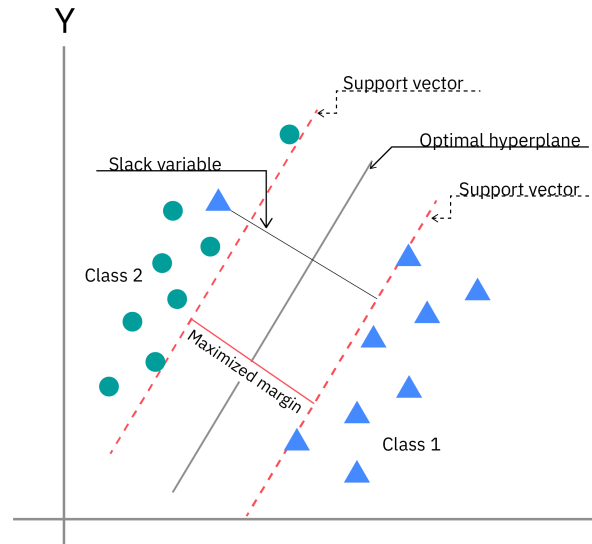


Ilustración 3.7: Representación del hiperplano de un SVM [39]

núcleo más empleadas es la función de base radial (RBF), propuesta por Broomhead y Lowe [40], que cuantifica la similitud entre vectores mediante una gaussiana:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2), \quad (3.14)$$

donde

- ✓ $x, x' \in R^d$ son vectores de entrada;
- ✓ $\|\cdot\|$ denota la norma euclídea;
- ✓ $\gamma > 0$ es un hiperparámetro que controla el ancho de la gaussiana (a mayor γ , menor ancho).

Diversos artículos han demostrado la eficacia del OC-SVM en tareas de detección de novedades en dominios prácticos. Yerima y Bashar [41] presentan un enfoque semi-supervisado basado en OC-SVM para la detección de spam en mensajes SMS, entrenando el modelo únicamente con ejemplos de mensajes legítimos. El sistema logra una tasa de detección del 100 % para mensajes spam con una tasa de falsos positivos inferior al 3 %, superando a enfoques supervisados tradicionales basados en bolsas de palabras con TF-IDF o frecuencia. Este trabajo demuestra cómo el OC-SVM puede adaptarse eficazmente a contextos donde la clase de interés (anomalía) es escasa o no etiquetada.

Por otro lado, Sadooghi y Khadem [42] proponen una mejora sistemática del esquema de detección de novedades con OC-SVM en señales de vibración de rodamientos. Su propuesta incluye una exhaustiva fase de preprocesamiento con técnicas no lineales como entropía de permutación y transformadas wavelet, así como una selección óptima del kernel y reducción de dimensionalidad mediante Sammon mapping. Los experimentos muestran que el uso de características no lineales extraídas del dominio tiempo-frecuencia mejora drásticamente el

rendimiento del OC-SVM, alcanzando hasta un 100% de precisión en detección de fallos mecánicos. Este estudio refuerza la importancia del diseño de características en sistemas de detección de novedades basados en OC-SVM.

Estos trabajos evidencian la versatilidad del OC-SVM para detectar patrones atípicos en contextos muy diversos, desde señales textuales hasta datos sensoriales complejos, lo cual resulta prometedor para su aplicación en el reconocimiento gestual en vídeos, donde los datos de entrenamiento suelen estar sesgados hacia clases conocidas y es crucial identificar gestos no observados previamente.

3.3.4. Triplet Loss y aprendizaje métrico

El entrenamiento por tripletas es una técnica de aprendizaje supervisado utilizada para aprender un espacio métrico donde los ejemplos de la misma clase se encuentran más cerca entre sí que de los de clases distintas. Fue introducido por Schroff et al. en el sistema Face-Net [43], y ha demostrado ser eficaz en tareas de verificación de identidad, recuperación de imágenes y clasificación abierta.

Esta técnica tiene como objetivo aprender un espacio de vectores de características tal que, muestras pertenecientes a la misma clase se proyecten cerca unas de otras, mientras que aquellas de clases distintas se separen por una distancia mínima.

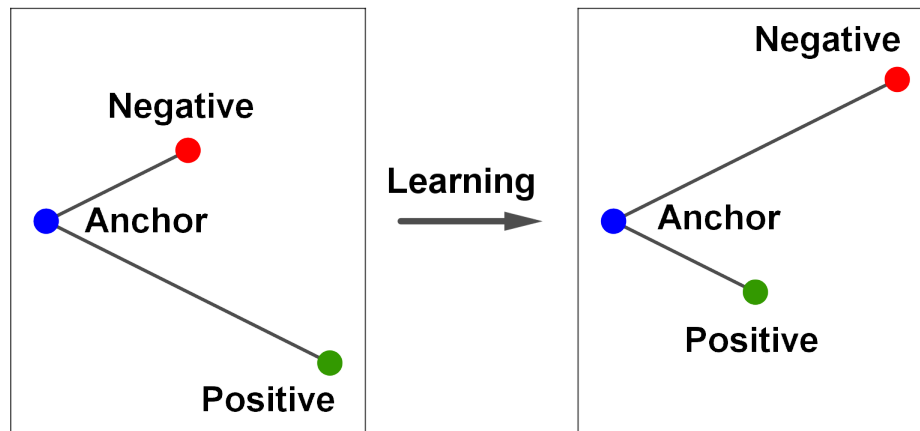


Ilustración 3.8: Representación del método Triplet Loss [44]

Cada muestra de entrenamiento se organiza en una tripleta, compuesta por una muestra de una clase denominada ancla, una muestra de la misma clase denominado positivo y una muestra de una clase diferente a la que pertenece el ancla y el positivo al que denominaremos como negativo. El objetivo del modelo es minimizar la distancia entre la muestra ancla y el positivo, y maximizar la distancia entre el ancla y el negativo, aplicando una función de pérdida conocida como *Triplet Loss*, que se define matemáticamente como:

$$L(a, p, n) = \max \{d(a, p)^2 - d(a, n)^2 + \alpha, 0\} \quad (3.15)$$

Donde:

- ✓ (a, p, n) : representa a la tripleta escogida.
- ✓ a : representa el ancla.
- ✓ p : representa el positivo.
- ✓ n : representa el negativo.
- ✓ $d(x, y)$: representa la métrica de distancia aplicada, en este caso se ha aplicado la euclídea.
- ✓ α : es un margen que impone una separación mínima entre las muestras de clases distintas.

Existen variantes del algoritmo de selección de tripletas como:

- ✓ **Batch-Hard**: selecciona las combinaciones más difíciles dentro de un minibatch, es decir, las negativas más cercanas y las positivas más alejadas, maximizando la dificultad del aprendizaje.
- ✓ **Semi-Hard**: selecciona negativas que están más alejadas que la positiva, pero aún dentro del margen de separación, lo que las convierte en ejemplos útiles para seguir afinando el espacio latente.

En el contexto del reconocimiento de gestos, el aprendizaje métrico mediante *triplet loss* permite mejorar la discriminación entre clases de gestos visualmente similares, y facilita la detección de gestos no vistos durante el entrenamiento. Esta capacidad de estructurar el espacio latente la convierte en una herramienta valiosa para escenarios de clasificación abierta.

3.3.5. Segmentación temporal de gestos

Una de las tareas más complejas en la interpretación automática de lengua de signos es la segmentación temporal, es decir, detectar con precisión el inicio y el fin de cada gesto dentro de un flujo continuo de vídeo. Esta tarea es fundamental en CSLR, ya que los gestos no están previamente segmentados ni delimitados.

A lo largo del tiempo, se han empleado distintas arquitecturas para abordar esta tarea. Los primeros enfoques se basaban en modelos ocultos de Markov, pero fueron progresivamente reemplazados por redes neuronales más complejas. Entre los modelos más destacados se encuentran:

- ✓ **CNN-LSTM-HMM** como el modelo de Koller et al. [45], que combina convoluciones espaciales, modelado temporal mediante LSTM y decodificación con HMM.

- ✓ **Encoder-Decoder con CTC**, como en el trabajo de Cui et al. [46], donde se emplean redes bidireccionales LSTM junto a la función de pérdida Connectionist Temporal Classification (CTC) para aprender alineaciones sin segmentación explícita.
- ✓ **Transformer-based Models**, como el enfoque de Zhou et al. [47], que utiliza mecanismos de atención para modelar dependencias globales en la secuencia.
- ✓ **Hybrid Models**, como el reciente trabajo de Min et al. [48], que aprovecha anotaciones temporales durante el entrenamiento para mejorar la segmentación.

Recientemente, se han explorado modelos basados en convoluciones temporales, como las Temporal Convolutional Networks (TCN), las cuales ofrecen una alternativa eficiente y eficaz frente a los modelos recurrentes. Los modelos TCN emplean convoluciones dilatadas y causales para capturar dependencias a largo plazo sin necesidad de recurrencia, lo que reduce el coste computacional y mejora la estabilidad del entrenamiento.

Temporal Convolutional Networks

Los modelos TCN son una arquitectura de redes neuronales profundas diseñada específicamente para procesar datos secuenciales. A diferencia de las redes recurrentes (como LSTM o GRU), que procesan secuencias de manera iterativa, las TCN aplican convoluciones unidimensionales sobre la dimensión temporal. Esta estrategia permite un procesamiento paralelo más eficiente y mejora la capacidad de capturar dependencias a largo plazo, aspecto fundamental en tareas donde las relaciones temporales distantes son clave.

Una TCN se compone de una pila de capas convolutivas 1D aplicadas jerárquicamente sobre la secuencia de entrada. En cada capa, se generan nuevas representaciones temporales donde la predicción en un instante t se basa en una ventana temporal que incluye múltiples pasos anteriores (y, opcionalmente, futuros), definidos por el campo receptivo de la red. Este campo crece con la profundidad de la red y el tamaño del filtro, lo que permite incorporar un contexto temporal cada vez más amplio.

Formalmente, dada una entrada $\mathbf{X} \in R^{T \times F}$, donde T es la longitud de la secuencia y F el número de características por paso, cada capa aplica una convolución causal o acausal, preservando la dimensión temporal mediante *padding* adecuado. El resultado es una secuencia del mismo tamaño, pero con representaciones más abstractas y contextualizadas, aprendidas a partir de patrones temporales.

En el trabajo de Bai et al. 'An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling' [49] se destacan varias propiedades clave de las TCN que las hacen competitivas frente a las redes recurrentes tradicionales. Estas incluyen:

- ✓ **Convoluciones causales** que garantizan que la salida en un instante dependa únicamente de entradas pasadas y presentes, asegurando coherencia temporal en tareas en tiempo real.
- ✓ **Convoluciones dilatadas** que permiten ampliar el campo receptivo de forma exponencial sin aumentar significativamente la profundidad ni el número de parámetros,

facilitando el modelado de dependencias a largo plazo.

- ✓ **Conexiones residuales** que mejoran la estabilidad del entrenamiento, permiten redes más profundas y mitigan el problema del desvanecimiento del gradiente.

Diversos experimentos han demostrado que las TCN ofrecen una generalización robusta, superando a modelos basados en RNN, para tareas de predicción de series temporales, el modelado del lenguaje o la síntesis musical. En el contexto específico de la segmentación y detección de acciones en vídeo, las TCN se han propuesto como una alternativa a las LSTM [50] y GRU [51], que tradicionalmente han sido empleadas para modelar dependencias temporales.

Numerosos trabajos han identificado limitaciones clave en el uso de modelos recurrentes. Pascanu et al. [52] explican que el entrenamiento de RNN profundas presenta problemas inherentes como el desvanecimiento y la explosión del gradiente, lo que dificulta la captura de dependencias temporales extensas. Además, Singh et al. [53] demuestran que los modelos LSTM tienden a cometer errores de sobresegmentación en tareas de detección de acciones finas, debido a una atención limitada a largo plazo y a su dependencia secuencial de los datos. Huang et al. [54], en un estudio sobre etiquetado de acciones con supervisión débil, observan que los modelos recurrentes requieren un gran volumen de anotaciones para alcanzar un desempeño competitivo, lo cual afecta su escalabilidad práctica.

Frente a estas limitaciones, las TCN se estructuran como redes convolucionales aplicadas sobre la dimensión temporal, con computaciones completamente paralelizables por capa. Esto permite prescindir del estado oculto recurrente y extender el campo receptivo a través de filtros de gran tamaño o dilataciones. El artículo de Lea et al. [55] presenta dos variantes de esta arquitectura: la *Encoder-Decoder TCN* (ED-TCN) y la *Dilated TCN* (D-TCN).

Encoder-Decoder Temporal Convolutional Network

La ED-TCN es una arquitectura de redes convolutivas temporales específicamente diseñada para tareas de segmentación y detección de acciones en vídeo, donde es crucial clasificar cada instante temporal según la acción que se está ejecutando.

A diferencia de una TCN genérica, ED-TCN adopta una estructura *encoder-decoder*, inspirada en arquitecturas de segmentación semántica en visión por computador, tal y como se puede observar en la Figura 3.9. Esta estructura no solo captura dependencias de largo alcance, sino que también modela transiciones temporales con mayor precisión, preservando la resolución temporal de la secuencia original.

El modelo se compone de dos bloques principales:

- ✓ **Codificador (Encoder)**: varias capas de convoluciones 1D seguidas de operaciones de *max pooling* que reducen progresivamente la resolución temporal. Cada capa aumenta la abstracción de las representaciones y amplía el campo receptivo.
- ✓ **Decodificador (Decoder)**: reconstruye una secuencia de salida de la misma longitud que la entrada, mediante operaciones de *upsampling* seguidas de nuevas convoluciones.

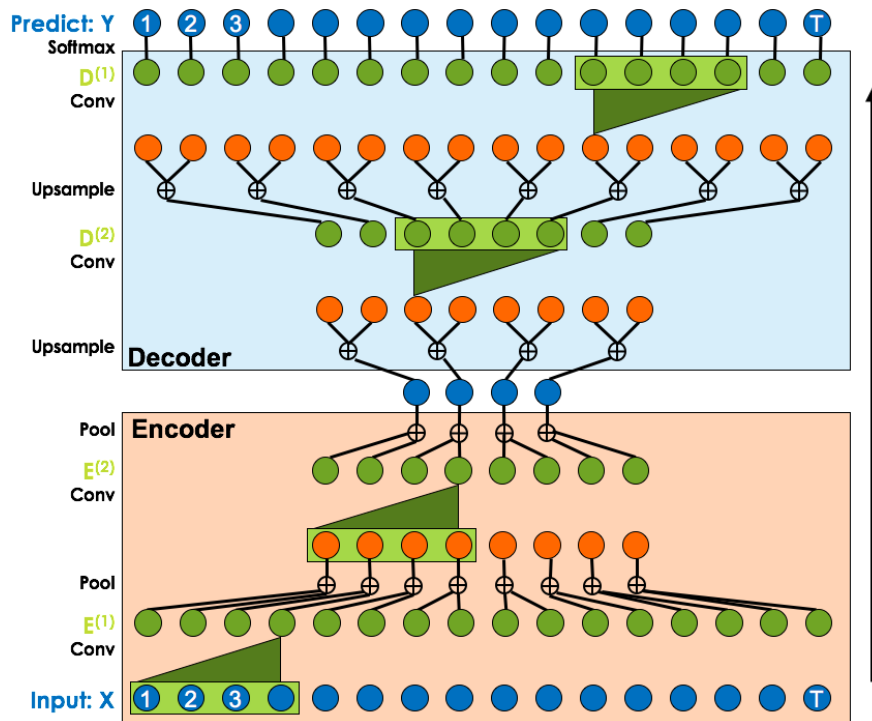


Ilustración 3.9: Representación de la arquitectura ED-TCN [55]

Su objetivo es recuperar la granularidad temporal y proporcionar una predicción por instante de tiempo.

Ambos bloques están conectados simétricamente, y suelen emplearse conexiones residuales o *skip connections* entre niveles correspondientes para facilitar el flujo de información y el entrenamiento.

Dilated Temporal Convolutional Network

Por otro lado, la arquitectura D-TCN es una red basada en convoluciones unidimensionales inspirada en modelos como WaveNet [56], que incrementa su campo receptivo usando factores de dilatación, sin necesidad de aumentar la profundidad de la red ni recurrir a operaciones de *pooling*. Gracias a esta propiedad, puede modelar dependencias de largo alcance en señales temporales manteniendo la resolución temporal original.

Su principio clave es el uso de convoluciones dilatadas, que introducen espacios entre los elementos del filtro, como se observa en la Figura 3.10, permitiendo abarcar una ventana temporal más amplia con menos capas.

La convolución dilatada introduce huecos entre las posiciones del filtro, de modo que un único kernel de longitud fija opera sobre muestras espaciadas d pasos. Esto se formaliza en la Ecuación (3.16) (versión no causal) y en la Ecuación (3.17) (versión causal). El resultado es un aumento lineal del campo receptivo con d para cada capa, sin modificar el número de parámetros.

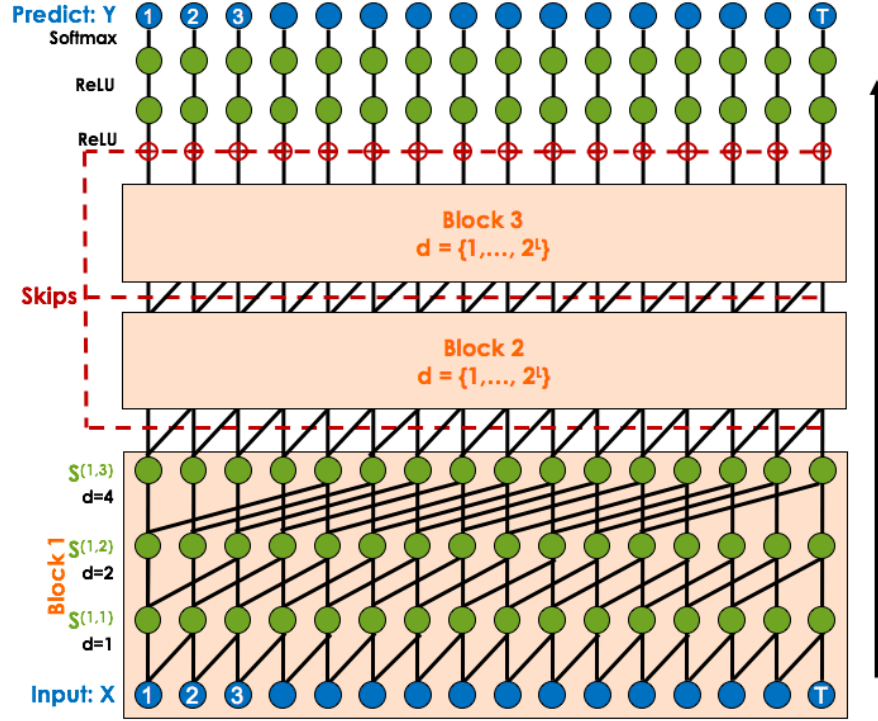


Ilustración 3.10: Representación de la arquitectura D-TCN [55]

$$y_t = (\mathbf{x} *_{\mathbf{d}} \mathbf{w})_t = \sum_{k=0}^{K-1} w_k x_{t-d(k-\lfloor (K-1)/2 \rfloor)} \quad (3.16)$$

Donde:

- ✓ y_t : activación (salida) de la operación convolucional en el instante t .
- ✓ $\mathbf{x} = \{x_t\}_{t \in \mathbb{Z}}$: señal o serie temporal de entrada; x_t es la muestra en el tiempo discreto t .
- ✓ $\mathbf{w} = \{w_k\}_{k=0}^{K-1}$: kernel convolucional de longitud K ; w_k es su k -ésimo coeficiente.
- ✓ $*_{\mathbf{d}}$: operador de convolución dilatada con factor d .
- ✓ $K \in \mathbb{N}$: tamaño (número de pesos) del filtro.
- ✓ $d \in \mathbb{N}$: factor de dilatación; fija la separación entre las posiciones del filtro.
- ✓ k : índice interno que recorre los coeficientes del kernel.
- ✓ $\lfloor \cdot \rfloor$: función suelo, utilizada para centrar simétricamente el filtro (caso no causal).

$$y_t = \sum_{k=0}^{K-1} w_k x_{t-dk}, \quad t \geq 0. \quad (3.17)$$

Donde:

- ✓ $t \geq 0$: dominio temporal restringido al presente y al pasado para preservar la causalidad.
- ✓ Las restantes variables y_t , w_k , x_t , K , d , k mantienen los significados definidos anteriormente.

La Ecuación (3.18) muestra que, para un filtro de tamaño K y un factor de dilatación d , la salida de la capa depende de $R = d(K - 1) + 1$ muestras de entrada. Así, con $K = 3$ y $d = 2$, cada salida considera cinco instantes diferentes, capturando contexto de rango más amplio que una convolución estándar.

$$R = d(K - 1) + 1. \quad (3.18)$$

Donde:

- ✓ R : *campo receptivo* de una única capa; número total de muestras de entrada que influyen en cada salida.
- ✓ K : longitud del filtro.
- ✓ d : factor de dilatación.

Apilar varias capas dilatadas con factores $\{d_\ell\}_{\ell=0}^{L-1}$ produce un campo receptivo global R_L dado por la Ecuación (3.19). Cuando se emplea la progresión geométrica $d_\ell = 2^\ell$ (estrategia típica de WaveNet), R_L crece exponencialmente con L , lo que permite cubrir horizontes temporales muy extensos con un número moderado de capas.

$$R_L = 1 + (K - 1) \sum_{\ell=0}^{L-1} \left(\prod_{j=0}^{\ell} d_j \right). \quad (3.19)$$

Donde:

- ✓ R_L : campo receptivo total tras apilar L capas dilatadas.
- ✓ $L \in \mathbb{N}$: número de capas de la red.
- ✓ K : longitud (común) del filtro en cada capa.
- ✓ d_ℓ : factor de dilatación en la capa ℓ ($\ell = 0$ es la capa más cercana a la entrada).
- ✓ d_j : símbolo de iteración interna dentro del producto; coincide con d_j .
- ✓ $\sum_{\ell=0}^{L-1} (\cdot)$: suma que acumula la contribución de cada capa.
- ✓ $\prod_{j=0}^{\ell} d_j$: producto de los factores de dilatación desde la capa 0 hasta la capa ℓ ; determina la expansión efectiva del campo receptivo hasta esa profundidad.

A igualdad de longitud de filtro K , la dilatación multiplica el campo receptivo por d con la misma complejidad $\mathcal{O}(K)$ y sin aumentar los parámetros aprendibles, ofreciendo una memoria temporal más larga con eficiencia de cómputo y de memoria.

Las Ecuaciones (3.16) (3.19) formalizan la operación de convolución dilatada y cuantifican la extensión del campo receptivo para arquitecturas TCN. Esta formulación explica por qué la *Dilated TCN* puede modelar dependencias de largo alcance sin profundizar en exceso la red ni degradar la resolución temporal de las activaciones.

Ambas variantes superan en rendimiento a los modelos LSTM bidireccionales evaluados en los mismos conjuntos de datos. Por ejemplo, en el dataset 50 Salads [57], las TCN alcanzan mejores resultados en métricas como F1 segmental y *segmental edit distance*, mientras reducen significativamente los errores de sobresegmentación que afectan a los modelos recurrentes. Además, los experimentos sintéticos incluidos en [55] demuestran que las TCN pueden capturar composiciones de acciones, duraciones de segmentos y patrones de transición complejos que los modelos recurrentes no logran representar adecuadamente.

3.3.6. Métricas de evaluación

Las métricas empleadas para la evaluación de los modelos han sido:

- ✓ **Tasa de acierto:** también conocida como *accuracy*, mide la proporción de predicciones correctas sobre el total de muestras.
- ✓ **Precisión:** indica la proporción de verdaderos positivos (TP) entre todos los casos clasificados como positivos por el modelo, ya sean verdaderos positivos o falsos positivos (FP).

$$\text{Precisión} = \frac{TP}{TP + FP}$$

- ✓ **Sensibilidad:** también llamada *recall*, mide la proporción de verdaderos positivos detectados correctamente sobre el total de positivos reales, es decir, aquellas muestras clasificadas como verdaderos positivos o como falsos negativos (FN).

$$\text{Sensibilidad} = \frac{TP}{TP + FN}$$

- ✓ **F1-score:** es una métrica balanceada entre la precisión y la sensibilidad, útil cuando se desea un compromiso entre ambas.

$$F1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Sensibilidad}}{\text{Precisión} + \text{Sensibilidad}}$$

- ✓ **UAR (Unweighted Average Recall):** es el promedio no ponderado de la sensibilidad obtenida para cada clase. Esta métrica es especialmente útil en contextos con clases

desbalanceadas, ya que da el mismo peso a todas las clases independientemente de su frecuencia.

$$\text{UAR} = \frac{1}{C} \sum_{i=1}^C \frac{TP_i}{TP_i + FN_i}$$

donde C es el número total de clases, y TP_i , FN_i son los verdaderos positivos y falsos negativos de la clase i , respectivamente.

Además, para la evaluación de modelos de detección de novedades, como el OC-SVM, se emplean métricas específicas que no dependen de un umbral fijo, lo que permite evaluar mejor su capacidad de discriminación.

- ✓ **AUROC (Área bajo la curva ROC)**: mide la capacidad del modelo para distinguir entre muestras positivas y negativas en distintos umbrales de decisión. Un valor de 1 indica una separación perfecta, mientras que 0.5 equivale a un modelo aleatorio
- ✓ **FPR@95 %TPR**: representa la tasa de falsos positivos cuando se alcanza una tasa de verdaderos positivos del 95 %. Es útil para evaluar el número de errores que el modelo comete al intentar garantizar una alta detección de positivos.
- ✓ **AUPR (Área bajo la curva Precisión-Sensibilidad)**: evalúa el compromiso entre precisión y sensibilidad, lo cual resulta más ilustrativo que el AUROC en escenarios con clases desbalanceadas.

Capítulo 4

Desarrollo

En este capítulo se expone la metodología escogida para el desarrollo de este trabajo de fin de grado, posteriormente se expondrán las fases de desarrollo en las que se ha dividido la elaboración del trabajo en base a la metodología.

4.1. Metodología

Para el desarrollo de este trabajo de fin de grado se ha propuesto seguir la metodología CRISP-DM (Cross-Industry Standard Process for Data Mining).

CRISP-DM es una metodología ampliamente aceptada en proyectos relacionados con la minería de datos y el aprendizaje automático, que sigue un ciclo compuesto por seis fases esenciales, detalladas a continuación:

- ✓ **Comprensión del negocio:** En esta primera fase se debe llevar a cabo un análisis exhaustivo para definir claramente los objetivos del proyecto. En el caso de este trabajo de fin de grado, los objetivos del proyecto están enfocados en investigar posibles soluciones que faciliten la interpretación de la lengua de signos mediante el uso de la inteligencia artificial y la visión por computador.
- ✓ **Comprensión de los datos:** Una vez definidos los objetivos, se procede a recopilar, explorar y realizar un análisis preliminar de los datos disponibles. En nuestro caso, el conjunto de datos empleados corresponde a un subconjunto de datos de libre disposición, creado por la Universidad de Vigo, permitiendo identificar los desafíos principales relacionados con la calidad y la variabilidad intrínseca en los datos obtenidos de intérpretes de lengua de signos.
- ✓ **Preparación de los datos:** Tras comprender los datos disponibles, se lleva a cabo una preparación y transformación de los mismos desde su formato original hacia uno más adecuado para su uso en modelos de inteligencia artificial. Para ello, se emplea la herramienta MediaPipe, con el objetivo de extraer puntos clave del cuerpo de los intérpretes a lo largo de secuencias de vídeo. Esta información permite la generación de

vectores de características capaces de representar patrones temporales útiles para los modelos predictivos.

- ✓ **Modelado:** En esta fase se diseñan, implementan y entrenan modelos de aprendizaje automático adecuados a los objetivos y datos del proyecto. Se han implementado específicamente modelos basados en RNN, tales como LSTM y GRU. Además, se han realizado experimentos exhaustivos variando configuraciones de hiperparámetros y técnicas complementarias, buscando optimizar el rendimiento en tareas de clasificación y detección automática de gestos, evaluando rigurosamente los modelos desarrollados, empleando métricas reconocidas como la tasa de acierto o exactitud (*accuracy*), precisión (*precision*), sensibilidad o exhaustividad (*recall*) y F1-score, que proporcionan una evaluación cuantitativa del desempeño y robustez de los modelos obtenidos.
- ✓ **Evaluación:** En esta penúltima fase se evaluarán los resultados obtenidos tras la fase de modelado y las conclusiones obtenidas a través de estos.
- ✓ **Despliegue:** Finalmente, se contempla la implementación práctica del sistema desarrollado, orientada a cumplir el objetivo general del proyecto. Sin embargo, en el marco de este trabajo de fin de grado, el despliegue completo en un entorno de producción real queda fuera del alcance definido inicialmente.

Como se observa en la Figura 4.1, las flechas de color negro indican las posibles transiciones entre fases mencionadas previamente, CRISP-DM habilita al usuario a retroceder a fases previas o avanzar a fases posteriores en base al estado del desarrollo del proyecto. Esto facilita revisar o corregir posibles inconvenientes detectados durante una de las fases y poder retroceder a fases previas con el objetivo de solventarlo.

La aplicación de la metodología CRISP-DM ha permitido un desarrollo ordenado y eficiente del proyecto, facilitando la integración progresiva de resultados parciales y permitiendo ajustes iterativos de acuerdo con la evolución del mismo.

4.2. Planteamiento del Proyecto

En esta sección se establece el contexto del proyecto, así como sus objetivos y alcance general, abordando además los posibles desafíos técnicos y metodológicos asociados a la detección y reconocimiento de gestos en la lengua de signos. Lo que correspondería con la fase de **Comprensión del negocio** de la metodología CRISP-DM.

El planteamiento inicial del proyecto implicó un acercamiento profundo y detallado al ámbito específico de la lengua de signos, ya que ni alumno ni tutores habían tenido previamente experiencia o formación específica en esta área. Debido a esta circunstancia, se organizó una reunión con el Aula de Lengua de Signos de la Universidad de Las Palmas de Gran Canaria, cuyo objetivo principal fue adquirir un entendimiento sólido del contexto, las particularidades y las necesidades prácticas relacionadas con la lengua de signos española.

Durante esta reunión se discutieron diversos aspectos fundamentales sobre la estructura y uso de la lengua de signos. Uno de los primeros puntos destacados fue la naturaleza diversa y

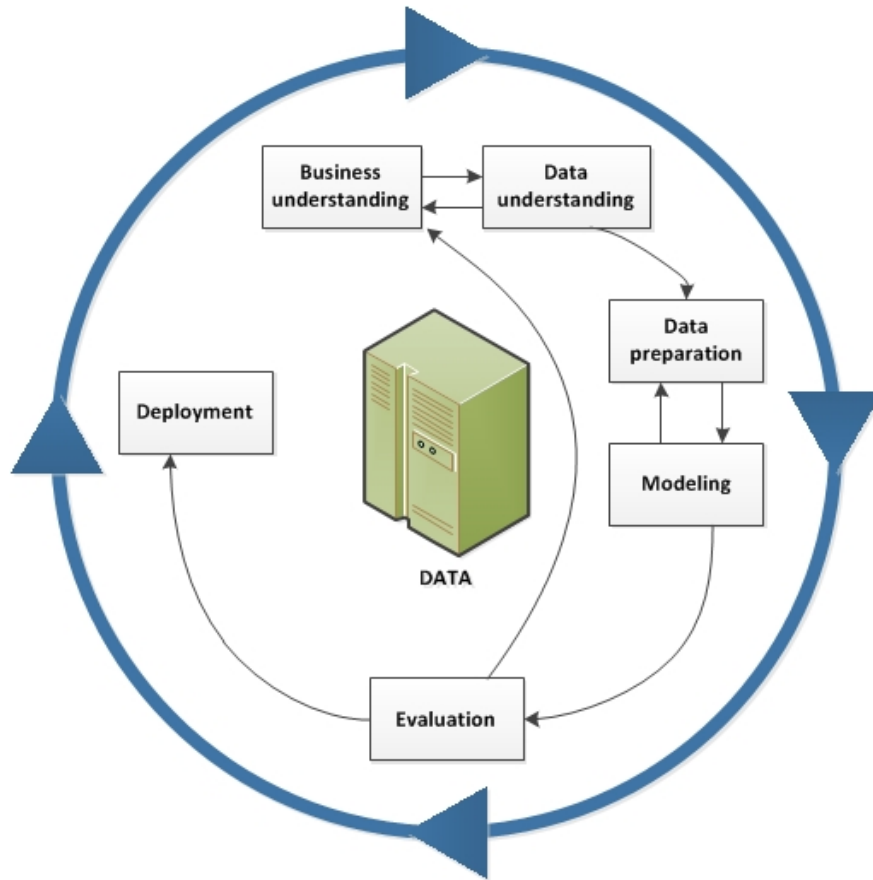


Ilustración 4.1: Ciclo de vida de CRISP-DM [58]

específica de las lenguas de signos, que varían según cada país, incluso cuando comparten el mismo idioma. Por ejemplo, países hispanohablantes como España y México tienen sistemas completamente independientes, denominados lengua de signos española (LSE) y lengua de señas mexicana (LSM), respectivamente. Debido a esta variabilidad, este trabajo de fin de grado se enfocará exclusivamente en la lengua de signos española.

Adicionalmente, se identificaron características clave de la lengua de signos española que impactan directamente en la metodología del proyecto:

- ✓ Los sustantivos propios generalmente se deletrean letra por letra, utilizando un alfabeto dactilológico.
- ✓ Los números se expresan mediante señas específicas.
- ✓ Cada gesto corresponde típicamente a una palabra o concepto particular.
- ✓ Las palabras suelen reducirse a su raíz o sustantivo común para simplificar y agilizar la comunicación.
- ✓ Los verbos normalmente se expresan en infinitivo.
- ✓ El orden sintáctico de las frases no está rígidamente establecido, ofreciendo una estruc-

tura flexible que depende principalmente del contexto comunicativo.

Estos aspectos de la lengua de signos representan un desafío significativo desde el punto de vista del aprendizaje automático, puesto que implican la necesidad de reconocer y distinguir gestos muy específicos, manejando además la variabilidad natural en su realización física.

4.3. Comprensión de los Datos

Los datos utilizados en este trabajo de fin de grado provienen del dataset LSE-Health-UVigo, el cuál es un conjunto de datos de libre acceso creado por la Universidad de Vigo. Este conjunto de datos está compuesto por 273 vídeos que presentan contenido médico interpretado en lengua de signos española por 10 intérpretes.

El dataset abarca un total aproximado de 11 horas de grabación y ha sido elaborado en un entorno controlado (Figura 4.2) utilizando un fondo croma azul, iluminación uniforme y sin sombras, grabado a una tasa de 25 fotogramas por segundo (FPS) y una resolución de *Full High Definition* (FHD), tal como se detalla en artículo de publicación del dataset [5]. Esto nos permite aplicar un tratamiento equitativo y uniforme al conjunto de muestras.

Cada vídeo cuenta con su correspondiente traducción al español a modo de segmentos, siguiendo la estructura de las frases empleadas por los intérpretes durante la grabación. En total, el dataset incluye 7.738 segmentos de frases, dentro de los cuales se han identificado y anotado 105 gestos diferentes a lo largo de los 273 vídeos disponibles, esto no implica que todos los gestos realizados por los intérpretes estén traducidos y tengan una anotación del momento en el que se realiza dicho gesto. Estas anotaciones generan un conjunto de 15.908 muestras individuales distribuidas entre las 105 clases de gestos.

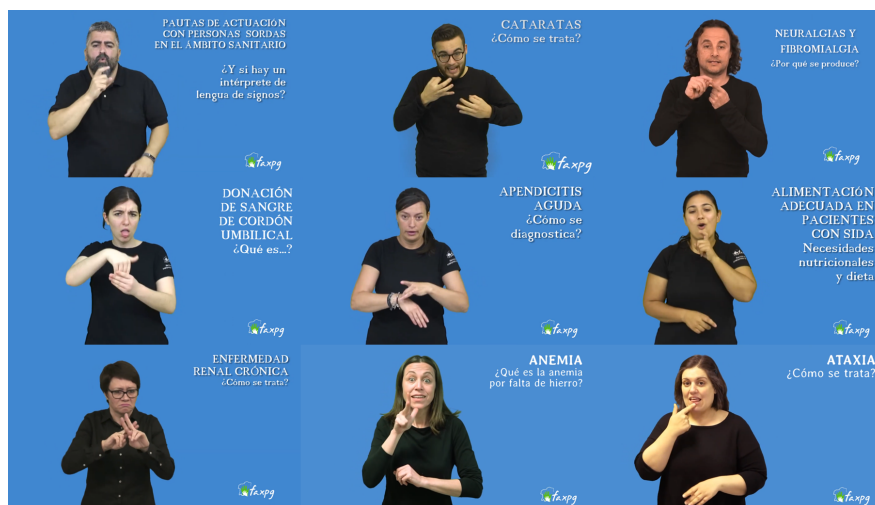


Ilustración 4.2: Muestra del conjunto de datos

Sin embargo, uno de ellos ha sido excluido debido a problemas con la realización de gestos de dicho intérprete. Esto se debe a que, este intérprete presentaba una postura inusual en una

de sus manos cuando no estaba involucrada en la ejecución del gesto, lo que podría introducir ruido o inconsistencias en el modelo. Esta situación se ilustra en la Figura 4.3.



Ilustración 4.3: Ejemplo del intérprete eliminado por adoptar una postura no natural

Tras eliminar a dicho intérprete el conjunto de muestras ha sido reducido en su tamaño, y por ende, el número de muestras también ha sido reducido. Dado que el dataset acaba de sufrir una gran pérdida de muestras, se ha decidido implementar un filtro, cuyo objetivo es el de seleccionar aquellas clases de gestos que cumplan el criterio de ser empleadas por un mínimo de 3 intérpretes distintos, y dichos gestos deben de haber sido realizados 5 o más veces, con un mínimo de 60 muestras totales entre los intérpretes que realicen dicho gesto.

Bag of Words

Para la implementación de dicho filtro se ha seguido la metodología de procesamiento de lenguaje natural **Bag Of Words** (BoW) [59]. La metodología BoW se basa en extraer las palabras de frases ignorando por completo el orden de aparición de dichas palabras, para enfocarnos en la frecuencia de aparición de las palabras. Obteniendo como resultado una representación vectorial de cada frase, donde cada dimensión de dicho vector representa a una palabra del vocabulario empleado en las frases, y el valor atribuido a dicha dimensión representaría el número de apariciones de dicha palabra en la frase.

A modo de ejemplo, se han preparado tres frases simples para poner en práctica la metodología BoW:

- ✓ Frase 1: El balón impactó contra la ventana.
- ✓ Frase 2: La casa tiene una ventana rota.
- ✓ Frase 3: Hemos perdido el balón.

A partir de las palabras que se encuentran en estas frases construiremos el BoW correspondiente que se muestra en la Tabla 4.1.

Cuadro 4.1: Representación Bag of Words de tres frases

Frase	balón	casa	contra	el	hemos	impactó	la	perdido	rota	tiene	una	ventana
1	1	0	1	1	0	1	0	0	0	0	0	1
2	0	1	0	0	0	0	1	0	1	1	1	1
3	1	0	0	1	1	0	0	1	0	0	0	0

En nuestro caso, se emplearán las transcripciones a modo de corpus, y se creará un BoW usando como vocabulario todas las palabras anotadas en dichas transcripciones, lo quiere decir que tenemos un vocabulario de tamaño 105, y por ende unas representaciones vectoriales de las transcripciones de dimensión 105.

Una vez implementado el BoW del conjunto de transcripciones, se agruparán dichas representaciones vectoriales de videos por intérprete, con el objetivo de conocer cuáles son las palabras empleadas por cada intérprete en el conjunto de videos en total. Tras ello se aplicará el filtro mencionado previamente, obteniendo un total de 43 clases de gestos que cumplen dicho requisito.

Esta reducción tan significativa de gestos es de esperar, teniendo en cuenta la temática médica del dataset. La amplia mayoría de videos se enfocan en enfermedades que afectan a un órgano en específico, y dichos videos siempre son interpretados por el mismo intérprete y suelen estar divididos en varias partes. Esto ha causado un aumento de muestras de un gesto en particular únicamente realizado por un mismo intérprete, incluso llegando al caso de solo tener muestras de un gesto de un único intérprete.

Una vez obtenidas las gestos a emplear, se comprobó el número de muestras que se habían obtenido por gesto. Para ello realizamos un análisis de distribución de datos por gesto, en la Figura 4.4 se muestran los resultados de dicho análisis, destacando el desbalanceo entre las muestras de gestos que concierne al conjunto de datos.

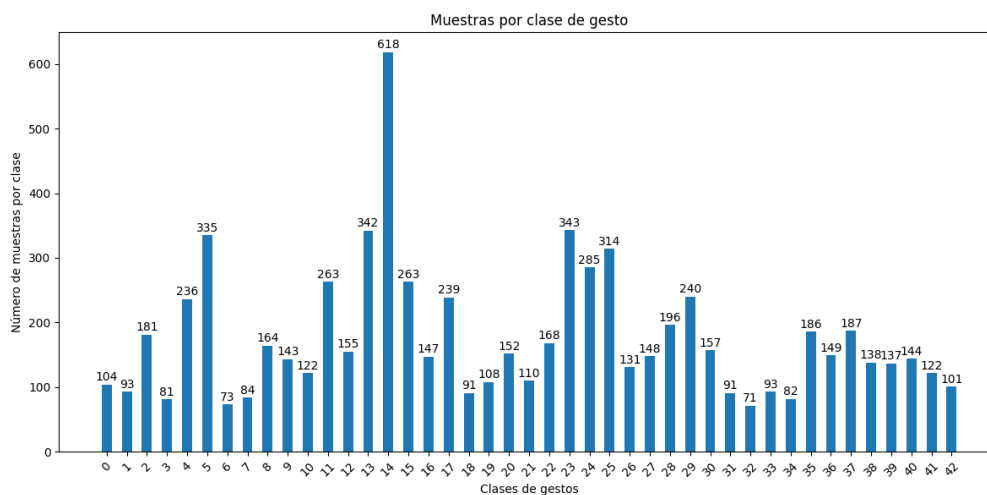


Ilustración 4.4: Representación del número de muestras por clase de gesto

Por otro lado cabe destacar que, al igual que ocurre en las lenguas orales, en la lengua

de signos cada intérprete posee un ritmo y estilo propios al ejecutar los gestos. Además, la postura corporal puede variar incluso cuando se realiza el mismo signo, lo que contribuye a una mayor variabilidad durante la gesticulación. Como consecuencia, el dataset cuenta con una considerable disparidad en la duración de los gestos, esta variabilidad se puede observar entre gestos de distinta clase y entre gestos de la misma clase. Tal como se muestra en la Figura 4.5, existen gestos que se extienden únicamente durante 8 fotogramas y otros que alcanzan hasta 50 fotogramas.

Es por ello, que en este dataset se cuenta con una gran disparidad entre las duraciones de los gestos en general, independientemente de si se trata del mismo gesto o de gestos diferentes. Como se observa en la Figura 4.5, tenemos gestos con una duración de 8 frames y gestos con una duración de 28 frames en un mismo dataset, lo que perjudicaría el proceso de aprendizaje de los modelos.

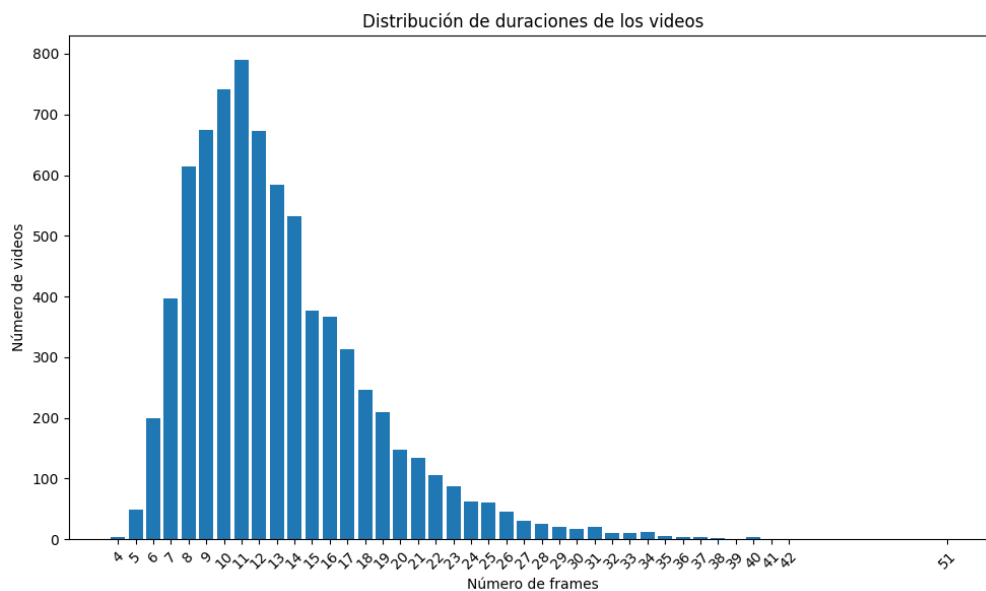


Ilustración 4.5: Gráfica de las duraciones de vídeos de gestos aislados

Esta inconsistencia en la duración de los gestos puede afectar negativamente el proceso de aprendizaje de los modelos, al dificultar la comparación y generalización entre ejemplos. Para mitigar este problema, se han considerado dos estrategias, una de ellas basada en el uso de relleno (*padding*) y otra basada en el muestreo.

Ambas opciones se explicarán en detalle en la subsección de **Procesamiento de los Datos**, y serán evaluadas con el objetivo de garantizar al mismo tiempo la preservación de la información relevante de los gestos y la coherencia temporal del conjunto de datos.

4.4. Procesamiento de los Datos

Tras establecer el conjunto de gestos que emplearemos para generar el dataset, proseguiremos con el procesamiento de los videos con el objetivo de extraer las características de los gestos. Para ello, nos enfocamos en identificar aquellas características que nos brinden un mayor conocimiento sobre los patrones o secuencias de movimiento ejecutadas por cada intérprete al realizar un determinado gesto.

Mediante el uso de las herramientas de visión por computador mencionadas en la subsección **Herramientas de Visión por Computador 3.3.1**, se extraerán las características de mayor relevancia de los videos. Para ello nos apoyaremos en el uso de OpenCV, para recortar los videos a los intervalos exactos donde se producen los gestos previamente filtrados. Posteriormente, extraemos los puntos de referencia frame a frame utilizando las soluciones de MediaPipe mencionadas. Estas características se almacenan en archivos CSV, donde cada fila representa un frame y las columnas contienen las coordenadas de las características extraídas para ese instante, en cada video.

Una vez extraídas todas las características de los videos, obtendremos un dataset compuesto de las coordenadas de las características de los intérpretes cuando estos realizan un gesto. A partir de estas coordenadas, se calculan métricas que servirán como base para el entrenamiento de los modelos, concretamente, las distancias entre puntos corporales relevantes para la interpretación.

Cálculo de distancias entre puntos de referencia

Las distancias entre los puntos comprenden relaciones entre partes del cuerpo que resultan esenciales en la ejecución de la mayoría de los gestos, como los dedos, muñecas, codos, hombros y cabeza. Además, nos ofrecen información relevante sobre la configuración corporal actual del intérprete: si la mano está abierta o cerrada, si el brazo está extendido o flexionado, entre otros aspectos.

De entre el amplio conjunto de puntos de referencia extraídos con MediaPipe, para el cálculo de distancias solo se considerarán aquellos correspondientes a: muñecas, puntas de los dedos y articulaciones metacarpofalángicas de los dedos, codos, hombros y cabeza.

El principal inconveniente del uso de distancias como métrica es la variabilidad morfológica entre intérpretes, quienes pueden presentar diferencias significativas en altura o complexión física. Esta variabilidad puede introducir una alta disparidad en los datos, lo que hace necesario aplicar una estrategia de normalización que se adecúe al problema que nos estamos enfrentando.

Por esta razón, se ha recurrido a la antropometría, la ciencia encargada de estudiar las dimensiones y proporciones del cuerpo humano, para la normalización de estas distancias. Según el estudio antropométrico del ejército de los Estados Unidos de América [60], la altura del torso, desde los hombros hasta la cadera, suele representar entre un 24 % y un 28 % de la estatura total de una persona adulta. Por tanto, se ha optado por utilizar el 26 % de la altura

corporal como valor de referencia para la normalización de las distancias entre los puntos de referencia.

Una vez aplicada esta regla de normalización, se obtiene un conjunto de datos que representa distancias relativas entre puntos de referencia, normalizadas respecto a la morfología individual de los intérpretes. Sin embargo, el hecho de tener datos normalizados no implica que el dataset esté listo para su uso en el entrenamiento de modelos de aprendizaje, ya que existe un segundo factor que introduce variabilidad en los gestos: la duración de los gestos.

Como se mencionó previamente, debido a la inconsistencia en la duración de los gestos, se han planteado dos estrategias con el objetivo de mitigar dicho problema:

Estrategia de relleno

El *padding* es una técnica utilizada en el procesamiento de secuencias para unificar la longitud de estas, esto se logra rellorando el final de las secuencias con menor duración al establecido con valores nulos o 0 a aquellas secuencias que tengan una longitud inferior a la máxima observada dentro del conjunto de datos.

Este enfoque permite preservar la secuencia original sin truncamientos, garantizando que no se pierda información relevante y facilitando la compatibilidad con redes neuronales recurrentes como LSTM o GRU. Además, esta técnica simplifica el preprocesamiento al eliminar la necesidad de recortes manuales o generación de subventanas, y permite que el modelo aprenda sobre secuencias completas utilizando mecanismos de enmascaramiento para ignorar los valores añadidos durante el *padding*.

Estrategia de muestreo

En la estrategia de muestreo, se emplea un enfoque basado en ventanas deslizantes. Las secuencias cuya longitud supera un umbral predefinido se segmentan en múltiples ventanas de tamaño fijo utilizando un desplazamiento constante (*stride*), lo que permite la generación de subsecuencias solapadas. Por el contrario, aquellas secuencias cuya longitud es inferior al umbral se conservan en su totalidad y se les aplica *padding* para alcanzar la longitud establecida.

Para determinar el umbral óptimo en la estrategia de muestreo, se ha analizado la distribución de longitudes de las secuencias en la Figura 4.5. En ella se observa que la mayoría de los gestos tienen una duración comprendida entre 9 y 13 frames, siendo 11 frames la longitud más frecuente entre muestras de todas las clases. Por este motivo, se ha seleccionado dicho valor como umbral de referencia para aplicar en la estrategia de muestreo.

Una vez aplicadas ambas estrategias, se obtienen dos conjuntos de datos en los que se ha eliminado la variabilidad en la duración de las muestras. Esto permite su uso directo en el entrenamiento de modelos de aprendizaje automático, garantizando una mejor estabilidad en el proceso de aprendizaje.

4.5. Modelado

4.5.1. Modelado para un clasificador de gestos aislados

En esta subsección se describe la implementación de los modelos empleados en el proyecto para la tarea de clasificación de gestos aislados. Dado que los datos utilizados representan secuencias temporales, compuestas por series de coordenadas extraídas de características de cada frame de un video aislado de un gesto, se ha considerado apropiado el uso de modelos diseñados específicamente para el entrenamiento de datos secuenciales.

Para este trabajo de fin de grado, se ha optado por el uso de los modelos LSTM y GRU, mencionados en la subsección **Aprendizaje profundo y modelado secuencial** (3.3.2, 3.3.2), ambos en su variante bidireccional. Esta configuración permite a los modelos procesar la secuencia de entradas en ambas direcciones, desde el inicio hasta el final y viceversa, esto resulta de gran ayuda en tareas donde el contexto futuro es igual de relevante que el contexto pasado, como es el caso de los movimientos a lo largo de un gesto.

Funciones de Optimización y Pérdida

Adentrándonos en el proceso de entrenamiento de los modelos, se debe de hablar de elementos fundamentales de estos, como es el caso de las funciones de optimización y de pérdida, las cuáles juegan un papel crucial en la convergencia y el rendimiento del modelo.

La función de optimización escogida para entrenar los modelos ha sido Adam (*Adaptive Moment Estimation*) [61]. Adam es una función de activación que destaca por su eficiencia y rapidez de convergencia en su uso para el aprendizaje de redes neuronales profundas. Esta función de optimización combina las ventajas de los algoritmos de optimización en los que está basado: AdaGrad [62] y RMSProp [63].

AdaGrad

AdaGrad (*Adaptive Gradient Algorithm*) es una función de optimización basada en la obtención de una tasa de aprendizaje adaptable para cada uno de los pesos del modelo, con el objetivo de realizar pequeñas actualizaciones de parámetros asociados con características que ocurren con frecuencia y actualizaciones más grandes para parámetros asociados con características que ocurren con poca frecuencia.

La regla de actualización de AdaGrad se basa en modificar la tasa de aprendizaje (η) en cada paso de tiempo (t) para cada parámetro (θ_i), en función de los gradientes anteriores para θ_i , tal y como se puede observar en las ecuaciones 4.2 y en el algoritmo 1.

$$G_{t,i} = \sum_{\tau=1}^t g_{\tau,i}^2 \quad (4.1)$$

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i} + \delta}} \cdot g_{t,i} \quad (4.2)$$

donde:

- ✓ η es la tasa de aprendizaje inicial.
- ✓ $g_{t,i}$ es el gradiente de la función de pérdida respecto al parámetro θ_i en el paso t .
- ✓ $G_{t,i}$ es la suma acumulada de los cuadrados del gradiente para el parámetro i .
- ✓ δ es un valor pequeño añadido para evitar divisiones por cero, generalmente $\delta = 10^{-8}$.

Algoritmo 1 Actualización de parámetros con AdaGrad

Require: Tasa de aprendizaje η , término de suavizado δ

Ensure: Parámetros optimizados θ

- 1: Inicializar $\theta \leftarrow$ valores iniciales
 - 2: Inicializar $G \leftarrow 0$ ▷ Vector del mismo tamaño que θ
 - 3: **for** $t = 1$ **to** T **do**
 - 4: $g \leftarrow \nabla_{\theta} L(\theta)$ ▷ Gradiente actual
 - 5: $G \leftarrow G + g \odot g$ ▷ Acumulación por componente
 - 6: $\theta \leftarrow \theta - \frac{\eta}{\sqrt{G+\delta}} \odot g$
 - 7: **end for**
 - 8: **return** θ
-

La principal limitación de AdaGrad ocurre tras realizar muchas iteraciones, el gradiente acumulado (G_t) crezca de forma monótona, lo que provoca que la división entre la tasa de aprendizaje y la raíz cuadrada del gradiente acumulado se vuelva minúscula, deteniendo el aprendizaje. Algoritmos posteriores como RMSProp introducen el decaimiento exponencial del gradiente acumulado (G_t), con el fin de evitar el detenimiento del aprendizaje.

RMSProp

RMSProp (Root Mean Square Propagation) es un algoritmo de optimización propuesto por Geoffrey Hilton [63] para mejorar el descenso por el gradiente. La idea principal tras esta función recae en ajustar la tasa de aprendizaje para cada peso individualmente, manteniendo una media móvil exponencial de los cuadrados de los gradientes para cada peso. Esto se utiliza para normalizar la magnitud de las actualizaciones de los pesos.

La regla de actualización de RMSProp se basa en calcular una media móvil del cuadrado de los gradientes para cada parámetro θ_i , utilizando un factor de decaimiento γ , tal y como se puede observar en las ecuaciones 4.4 y el algoritmo 2.

$$E[g^2]_{t,i} = \gamma E[g^2]_{t-1,i} + (1 - \gamma)g_{t,i}^2 \quad (4.3)$$

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{E[g^2]_{t,i} + \delta}} \cdot g_{t,i} \quad (4.4)$$

donde:

- ✓ η es la tasa de aprendizaje inicial.
- ✓ $g_{t,i}$ es el gradiente de la función de pérdida respecto al parámetro θ_i en el paso t .
- ✓ $E[g^2]_{t,i}$ representa la media móvil exponencial del cuadrado del gradiente para el parámetro θ_i .
- ✓ γ es el coeficiente de decaimiento.
- ✓ δ es un valor pequeño añadido para evitar divisiones por cero, generalmente $\delta = 10^{-8}$.

Algoritmo 2 Actualización de parámetros con RMSProp

Require: Tasa de aprendizaje η , coeficiente de decaimiento γ , término de suavizado δ

Ensure: Parámetros optimizados θ

- 1: Inicializar $\theta \leftarrow$ valores iniciales
 - 2: Inicializar $E \leftarrow 0$ ▷ Vector del mismo tamaño que θ
 - 3: **for** $t = 1$ **to** T **do**
 - 4: $g \leftarrow \nabla_{\theta} L(\theta)$ ▷ Gradiente actual
 - 5: $E \leftarrow \gamma \cdot E + (1 - \gamma) \cdot g \odot g$ ▷ Media móvil exponencial
 - 6: $\theta \leftarrow \theta - \frac{\eta}{\sqrt{E + \delta}} \odot g$
 - 7: **end for**
 - 8: **return** θ
-

RMSProp se desarrolló como una mejora directa de AdaGrad. A diferencia de este último, evita que la tasa de aprendizaje disminuya en exceso gracias al uso del promedio móvil en lugar de una suma acumulativa. Esto lo hace especialmente útil en el entrenamiento de redes neuronales profundas y recurrentes, donde las actualizaciones pequeñas y adaptativas son clave para una convergencia estable y eficiente.

Adam

Adam es un algoritmo de optimización propuesto por Kingma y Ba [61] que combina las ideas de AdaGrad y RMSProp. Su funcionamiento se basa en calcular promedios móviles de primer y segundo orden, media y varianza de los gradientes, que son corregidos por un sesgo, para obtener estimaciones más precisas durante las primeras etapas del entrenamiento. Estas estimaciones se utilizan para adaptar de forma individual la tasa de aprendizaje de cada parámetro 4.9 3.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (4.5)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (4.6)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (4.7)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (4.8)$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} * \hat{m}_t \quad (4.9)$$

Donde:

- ✓ m_t y v_t representan a un promedio exponencialmente decreciente de gradientes pasados y de gradientes cuadrados pasados respectivamente.
- ✓ β_1 y β_2 son factores de decaimiento que controlan la actualización de los momentos.
- ✓ $\hat{m}_t$ y $\hat{v}_t$ representan a las estimaciones de primer y segundo momento corregidas por sesgo, que se emplean para contrarrestar el sesgo hacia cero que poseen m_t y v_t al inicializar o cuando los factores de decaimiento son pequeñas, es decir, cuando están cerca de 1.
- ✓ θ_t son los parámetros del modelo en el instante t .
- ✓ α es la tasa de aprendizaje.
- ✓ ϵ es un valor pequeño para evitar divisiones por cero.

Algoritmo 3 Actualización de parámetros con Adam

Require: Tasa de aprendizaje α , coeficientes β_1, β_2 , término de suavizado ϵ

Ensure: Parámetros optimizados θ

```

1: Inicializar  $m \leftarrow 0$ ,  $v \leftarrow 0$ ,  $t \leftarrow 0$ 
2: for  $t = 1$  to  $T$  do
3:    $g \leftarrow \nabla_{\theta} L(\theta)$  ▷ Gradiente actual
4:    $m \leftarrow \beta_1 \cdot m + (1 - \beta_1) \cdot g$ 
5:    $v \leftarrow \beta_2 \cdot v + (1 - \beta_2) \cdot g \odot g$ 
6:    $\hat{m} \leftarrow \frac{m}{1 - \beta_1^t}$ 
7:    $\hat{v} \leftarrow \frac{v}{1 - \beta_2^t}$ 
8:    $\theta \leftarrow \theta - \frac{\alpha}{\sqrt{\hat{v}} + \epsilon} \odot \hat{m}$ 
9: end for
10: return  $\theta$ 

```

El motivo por el que se ha optado por emplear Adam en lugar de otras funciones de activación, recae en la capacidad que posee Adam para ajustarse dinámicamente a los cambios en la escala de los gradientes, lo que le otorga una gran ventaja en tareas de clasificación secuencial frente a otras funciones de activación.

Cross Entropy

En el caso de la función de pérdida, en una primera instancia se optó por la función de entropía cruzada (*Cross Entropy Loss*), ampliamente empleada en tareas de clasificación multiclase. La función de pérdida *Cross Entropy* se centra en medir la discrepancia entre la distribución de la probabilidad real y la distribución de predicha por el modelo.

Su formulación para un único caso es:

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^C y_i \log(\hat{y}_i) \quad (4.10)$$

Donde:

- ✓ C es el número total de clases.
- ✓ y_i es un valor binario (0 o 1) que indica si la clase i es la verdadera.
- ✓ $\hat{y}_i$ es la probabilidad predicha para la clase i .

Esta función penaliza fuertemente las predicciones incorrectas que asignan baja probabilidad a la clase verdadera, lo cual la convierte en una opción adecuada para tareas de clasificación supervisada.

Focal Loss

Por otro lado, se implementó la función de pérdida *Focal Loss* [64], una variante de la entropía cruzada diseñada específicamente para abordar problemas de desbalanceo de clases. Esta función introduce un factor de modulación que reduce la contribución de ejemplos correctamente clasificados, y un parámetro de enfoque para el entrenamiento de muestras que presenten disparidad en la dificultad de las muestras.

La función de pérdida *Focal Loss* se define como:

$$\mathcal{L}_{\text{FL}} = -\alpha_i(1 - \hat{y}_i)^\gamma \log(\hat{y}_i) \quad (4.11)$$

Donde:

- ✓ $\hat{y}_i$ representa la probabilidad predicha para la clase verdadera.
- ✓ α_i es un factor de ponderación para la clase i , utilizado para reforzar el aprendizaje en clases minoritarias.
- ✓ γ es el parámetro de enfoque, el cual ajusta el grado de penalización para ejemplos correctamente clasificados.

Al emplear *Class Weights* en las funciones *Cross Entropy* y *Focal Loss* se obtuvieron grandes mejoras en los entrenamientos de los modelos que se ven reflejados en las evaluaciones de estos. Cabe destacar que al emplear *class weights* como valor α en la función *Focal Loss*, se obtuvieron pequeñas mejoras en comparación al uso previo de *Cross Entropy*, debido a que se enfoca en mayor medida en el desbalanceo de clases.

Ponderación de Clases

Como se mostró en la Figura 4.4 en la subsección de **Comprensión de los Datos 4.3**, el dataset presenta grandes casos de desbalanceo de muestras de gestos. Lo que puede llegar a causar un sesgo hacia los gestos con una mayor representación en el conjunto de entrenamiento.

Con el objetivo de mitigar este problema, se implementó una estrategia de ponderación de clases (*Class Weights*), la cual consiste en asignar un mayor peso a las clases minoritarias del conjunto de entrenamiento. Esto se refleja directamente en el cálculo de la función de pérdida, donde los errores cometidos sobre clases minoritarias tienen un mayor impacto en la actualización de los pesos del modelo.

Evaluación de clasificadores de gestos aislados

En esta subsección se analizan los resultados obtenidos de la evaluación de los modelos de clasificación implementados durante la fase de modelado. Como se mencionó anteriormente, se utilizaron dos conjuntos de datos diferentes para gestos aislados, generados según las estrategias propuestas de *padding* y muestreo. Para cada entrenamiento se realizó una partición del conjunto de datos donde, el 70 % de las muestras formaron parte del conjunto de muestras de entrenamiento, un 15 % del total fue destinado al conjunto de muestras de validación, y el 15 % restante fue destinado al conjunto de muestras de evaluación. Cabe de recalcar, que para cada entrenamiento y evaluación se emplearon las mismas muestras tanto de entrenamiento, validación y evaluación, en base a la estrategia empleada.

Los experimentos descritos fueron realizados en un equipo con arquitectura x86_64, capaz de operar en modos de 32 y 64 bits. El sistema cuenta con dos procesadores Intel Xeon Silver 4110 a 2.10 GHz, con un total de 16 núcleos físicos y 32 hilos de ejecución distribuidos en dos sockets. El equipo dispone de 256 GB de memoria RAM DDR4, de los cuales más de 240 GB estaban disponibles para el procesamiento durante los experimentos. En cuanto a las unidades de procesamiento gráfico, se emplearon cuatro tarjetas NVIDIA GeForce GTX 1080 Ti, cada una con 11 GB de memoria GDDR5X.

Evaluación de modelos entrenados con el conjunto de padding

En primer lugar, abordamos los resultados obtenidos utilizando el conjunto de datos generado mediante la estrategia de *padding*. Con este conjunto, entrenamos dos tipos de redes neuronales recurrentes: una red GRU bidireccional (Bi-GRU) y una red LSTM bidireccional

(Bi-LSTM). Cada modelo fue entrenado utilizando diferentes funciones de pérdida previamente mencionadas: *Cross-Entropy* y *Focal Loss*, además de experimentar con diferentes valores del parámetro de enfoque γ . Los resultados obtenidos tras el entrenamiento de estos modelos se presentan en la Tabla 4.2

Cuadro 4.2: Evaluación de modelos con el conjunto de relleno. Los mejores resultados están en negrita.

Modelo	Función de pérdida	Accuracy (%)	F1-score (%)	UAR (%)
Bi-GRU	Cross-Entropy	76,8	74,0	75,3
Bi-GRU	Focal Loss ($\gamma = 2$)	75,7	73,0	75,6
Bi-GRU	Focal Loss ($\gamma = 3$)	73,4	71,0	73,3
Bi-LSTM	Cross-Entropy	75,5	73,0	73,3
Bi-LSTM	Focal Loss ($\gamma = 2$)	73,4	71,0	73,7
Bi-LSTM	Focal Loss ($\gamma = 3$)	79,8	78,0	80,2
Bi-LSTM	Focal Loss ($\gamma = 3,5$)	76,1	74,0	76,1

De acuerdo con la Tabla 4.2, que presenta los resultados del entrenamiento de los modelos que han sido entrenados mediante el conjunto de datos generado a partir de la estrategia de relleno, se observa que la mejor tasa de acierto obtenida fue del 78,7%, correspondiente al modelo Bi-LSTM entrenado con la función de pérdida *Focal Loss* con $\gamma = 3$. Este resultado destaca por encima de las demás combinaciones, indicando que un valor más alto del parámetro podría mejorar la clasificación al enfatizar el aprendizaje de las clases más difíciles o menos frecuentes.

Por otro lado, se observa que los modelos entrenados con la función de pérdida *Cross Entropy* obtuvieron resultados similares, aunque ligeramente inferiores al mejor resultado obtenido con *Focal Loss* y $\gamma = 3$.

En términos comparativos entre las arquitecturas GRU y LSTM, no se observa una diferencia marcada en cuanto al rendimiento global. Ambas redes mostraron resultados aceptables, aunque la LSTM con un parámetro ajustado destacó por encima de GRU.

Evaluación de modelos entrenados con el conjunto de muestreo

A continuación, se analizan los resultados obtenidos al utilizar el conjunto de datos generado mediante la estrategia de muestreo. Este conjunto se empleó para entrenar dos arquitecturas de redes neuronales recurrentes Bi-GRU y una Bi-LSTM. Al igual que en la evaluación anterior, se entrenaron ambos modelos con las funciones de pérdida *Cross Entropy* y *Focal Loss*, considerando diferentes valores del parámetro γ .

La Tabla 4.3 presenta los resultados obtenidos para cada combinación de modelo y función de pérdida utilizada:

Según los resultados obtenidos en la Tabla 4.3, el mejor desempeño corresponde al modelo Bi-LSTM con la función de pérdida *Focal Loss* con un valor de $\gamma = 2$, alcanzando una tasa de acierto del 77,8%. Este hallazgo sugiere que una penalización moderada en muestras bien

Cuadro 4.3: Evaluación de modelos con el conjunto de muestreo. Los mejores resultados están en negrita.

Modelo	Función de pérdida	Accuracy (%)	F1-score (%)	UAR (%)
Bi-GRU	Cross-Entropy	76,8	74,0	75,3
Bi-GRU	Focal Loss ($\gamma = 2$)	73,3	71,0	72,8
Bi-GRU	Focal Loss ($\gamma = 3$)	72,3	70,0	72,9
Bi-LSTM	Cross-Entropy	75,6	73,0	74,2
Bi-LSTM	Focal Loss ($\gamma = 2$)	77,8	76,0	77,7
Bi-LSTM	Focal Loss ($\gamma = 2,5$)	72,1	70,0	72,0
Bi-LSTM	Focal Loss ($\gamma = 3$)	72,9	71,0	73,8

clasificadas, proporcionada por un valor intermedio de γ , mejora la discriminación de las clases más complejas.

Además, se observa que los modelos entrenados con *Cross Entropy* también obtienen resultados consistentes, siendo el modelo Bi-LSTM el más destacado con una tasa de acierto del 76,7%, ligeramente inferior al mejor resultado alcanzado con *Focal Loss*.

Se observa también una disminución en el rendimiento al aumentar el valor del parámetro a 3 en ambos tipos de redes (GRU y LSTM), indicando que un incremento excesivo del énfasis sobre clases difíciles puede perjudicar la capacidad general del modelo para clasificar correctamente.

En términos generales, los resultados obtenidos con el conjunto de muestreo muestran patrones coherentes con los hallazgos anteriores en la evaluación basada en *padding*, donde la arquitectura Bi-LSTM combinada con la función de pérdida *Focal Loss* resulta particularmente eficaz para mejorar el rendimiento del modelo.

A modo de resumen, los resultados de ambas evaluaciones indican que la arquitectura Bi-LSTM combinada con la función de pérdida *Focal Loss* proporciona un desempeño consistentemente superior en la clasificación de gestos aislados, tanto en el conjunto basado en *padding* como en el conjunto generado mediante muestreo.

4.5.2. Detección de gestos continuos en videos

El reconocimiento de gestos aislados representa un avance significativo en el campo de la interpretación automática de la lengua de signos. No obstante, la comunicación real en lengua de signos se produce de forma continua, sin pausas evidentes entre gestos, lo que impone un desafío mayor: detectar y reconocer gestos individuales dentro de secuencias prolongadas y fluidas.

Para ello, se reutilizaron los modelos de clasificación entrenados previamente, seleccionando aquellos que habían alcanzado el mejor rendimiento durante las fases de evaluación. La hipótesis que motivó este experimento derivó del hecho de que los gestos presentes en vídeos continuos son, en esencia, los mismos que se utilizaron durante el entrenamiento en formato

aislado. Por consiguiente, se planteó que los modelos preentrenados podrían reconocer dichos gestos, si se lograban extraer ventanas temporales que contuvieran de forma suficientemente representativa a dichos gestos.

La metodología empleada consistió en una estrategia de ventanas deslizantes aplicadas sobre la secuencia completa del vídeo. Concretamente, se extrajeron subsecuencias de 11 y 51 fotogramas, coincidiendo con las longitudes utilizadas durante el entrenamiento de los modelos basados en muestreo y *padding*, respectivamente.

Estas ventanas se desplazaron con un cierto solapamiento a lo largo de cada vídeo, generando múltiples fragmentos temporales. Cada uno de ellos fue procesado individualmente mediante los clasificadores preentrenados, obteniendo una predicción por ventana.

Sin embargo, los resultados empíricos obtenidos demostraron que este enfoque no es eficaz en el contexto de secuencias gestuales continuas, obteniendo un **0.01** % de tasa de acierto. En la mayoría de los casos, las ventanas extraídas no coincidían con los gestos completos, sino que capturaban únicamente fragmentos, ya fuera el inicio, el final o una transición entre gestos consecutivos.

Como resultado, las predicciones fueron altamente inconsistentes, con una distribución de clases aparentemente aleatoria y una baja confianza del modelo. En otras palabras, el modelo no lograba inferir con precisión la categoría gestual si la entrada no correspondía de manera clara a un gesto completo y aislado, como sí ocurría durante su fase de entrenamiento

Dicha inconsistencia puede deberse, en gran medida, a la aparición de predicciones erróneas provocadas por gestos que no pertenecen al conjunto de entrenamiento, así como por la presencia de gestos incompletos fragmentados por los bordes de las ventanas o por gestos conocidos que aparecen solapados entre sí.

4.5.3. Detección de novedades en gestos

Tras completar el experimento sobre detección de gestos en vídeos, se decidió profundizar en la rama de detección de novedades [35]. Esta técnica del aprendizaje automático supervisado o no supervisado busca identificar muestras que no pertenecen a ninguna clase observada previamente durante el entrenamiento del modelo.

El objetivo principal de esta etapa experimental es evaluar si los modelos Bi-LSTM entrenados durante este trabajo de fin de grado han capturado adecuadamente la distribución subyacente de las clases observadas durante su entrenamiento. Para realizar esta evaluación, se llevará a cabo un experimento específico en el que dichos modelos deberán distinguir entre gestos aislados conocidos y gestos nuevos, no observados previamente.

Este planteamiento permite verificar no solo la capacidad del modelo para clasificar correctamente gestos conocidos, sino también para reconocer de manera efectiva patrones novedosos que no han formado parte del conjunto de entrenamiento, mostrando así una adecuada generalización y capacidad de autoconocimiento.

Para ello se propone emplear un codificador basado en un modelo Bi-LSTM preentrenado. Este codificador permitirá proyectar los gestos en un espacio latente donde puedan aplicarse técnicas de detección de novedades, como se detalla a continuación.

Codificador LSTM

El codificador se construye a partir de un modelo Bi-LSTM previamente entrenado, al que se le elimina su capa final completamente conectada. Así, se obtiene una función de codificación que proyecta cada gesto en un vector latente representativo.

$$E : R^{T \times d} \longrightarrow R^h,$$

donde:

- ✓ T es el número de pasos de tiempo (longitud de la ventana de entrada).
- ✓ d es la dimensión de las características de cada paso de tiempo.
- ✓ h es la dimensión del espacio latente (tamaño del vector de características).

Sea

$$x = (x_1, x_2, \dots, x_T) \in R^{T \times d}$$

una ventana de entrada. El codificador produce el vector de características latente

$$e = E(x) \in R^h, \quad (4.12)$$

donde cada componente e_j captura información semántica del contenido gestual en la ventana x .

Para cada una de las K clases conocidas $c \in \{1, \dots, K\}$, definimos un *prototipo vectorial* $p_c \in R^h$ como la media aritmética de los vectores de características de todas las muestras de la clase c en el conjunto de entrenamiento:

$$p_c = \frac{1}{N_c} \sum_{i=1}^{N_c} e_i \quad (4.13)$$

donde:

- ✓ N_c es el número de ventanas de entrenamiento cuya etiqueta es c .
- ✓ $e_i = E(x^{(i)})$ es el vector de características de la i -ésima ventana de la clase c .

Una vez obtenidos los prototipos $\{p_1, \dots, p_K\}$, caracterizamos cada nuevo vector de características e por su *vector de similitudes*

$$f(e) = [f_1(e), f_2(e), \dots, f_K(e)]^\top \in R^K,$$

donde cada componente $f_c(e)$ mide la *similitud coseno* entre e y el prototipo p_c :

$$f_c(e) = \frac{e \cdot p_c}{\|e\| \|p_c\|} = \frac{\sum_{j=1}^h e_j (p_c)_j}{\sqrt{\sum_{j=1}^h e_j^2} \sqrt{\sum_{j=1}^h (p_c)_j^2}}, \quad (4.14)$$

con:

- ✓ $e \cdot p_c = \sum_{j=1}^h e_j (p_c)_j$ el producto escalar en R^h .
- ✓ $\|e\| = \sqrt{e \cdot e} = \sqrt{\sum_{j=1}^h e_j^2}$ la norma euclidiana de e .
- ✓ $\|p_c\| = \sqrt{p_c \cdot p_c}$ la norma euclidiana de p_c .

Este vector $f(e)$ puede usarse directamente para tareas de clasificación, comparándolo con un umbral o pasándolo por una capa *softmax* si se desea normalizar las similitudes a una distribución de probabilidad.

Las muestras empleadas para extraer los vectores de características provienen del mismo conjunto de datos utilizado para entrenar el modelo original. Estos vectores de características son empleados posteriormente para entrenar un modelo OC-SVM similar al mencionado en la subsección **Detección de Novedades 3.3.3**.

Para nuestra situación, trataremos a todas las muestras aprendidas por el modelo como la clase conocida, mientras que todas las muestras de clases desconocidas se considerarán la clase desconocida.

Por ello, nuestro modelo OC-SVM se entrena específicamente sobre los vectores de similitud $f(e)$ generados por el codificador Bi-LSTM y los prototipos de clase.

Evaluación OC-SVM

Finalmente, se exploraron diferentes combinaciones de hiperparámetros:

- ✓ $\nu \in 0,01, 0,05, 0,1, 0,2$: parámetro que controla la fracción esperada de errores de clasificación.
- ✓ $\gamma \in 10^{-3}, 10^{-2}, 10^{-1}, 1$: controla la sensibilidad del núcleo RBF.

El mejor rendimiento en validación se obtuvo con los valores ($\nu = 0,01$, $\gamma = 1$).

Cuadro 4.4: Rendimiento del modelo OC-SVM

Métrica	Valor
AUROC	0.620
FPR@95 %TPR	0.816
AUPR	0.359

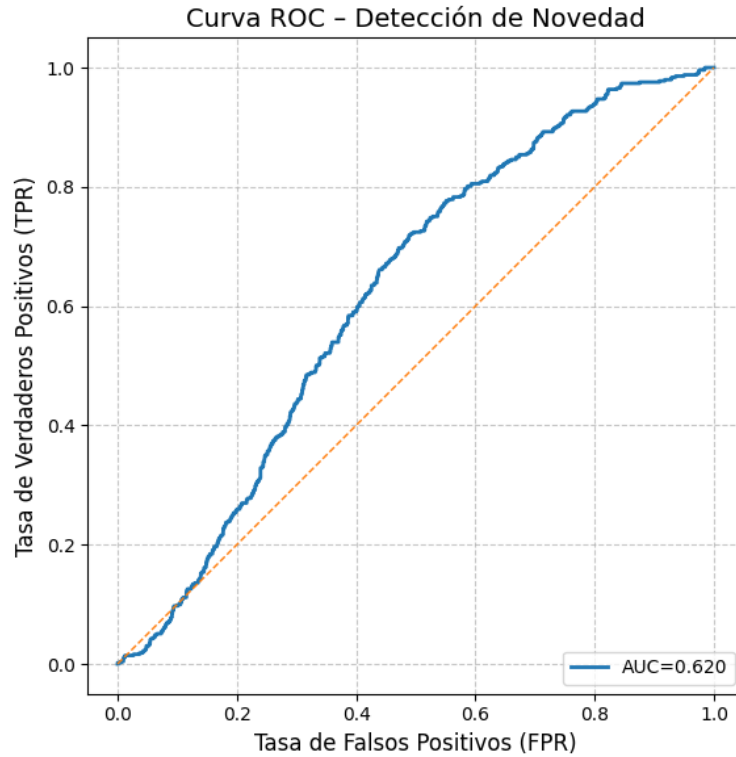


Ilustración 4.6: Representación de la curva ROC de OC-SVM

La curva ROC (Figura 4.6) ilustra el compromiso entre la tasa de verdaderos positivos (TPR) y la tasa de falsos positivos (FPR) a distintos umbrales de decisión. El valor del área bajo la curva ROC (AUROC) obtenido es de 0.353, indicando que el modelo tiene un rendimiento muy limitado.

El valor $\text{FPR}@95\% \text{TPR}$ de 0.816 implica que, para lograr identificar correctamente el 81% de los gestos conocidos, el modelo clasifica incorrectamente el 81.6% de los gestos novedosos como conocidos, mostrando una pobre capacidad discriminativa.

El valor AUPR de 0.359 refleja que el modelo tiene una muy baja capacidad para identificar correctamente los gestos novedosos.

En conclusión, los experimentos realizados para la detección de novedades en gestos han demostrado que, aunque los modelos Bi-LSTM empleados en este trabajo de fin de grado muestran un rendimiento adecuado en tareas de clasificación supervisada de gestos aislados conocidos, presentan limitaciones significativas cuando se enfrentan al reconocimiento de gestos novedosos o no observados previamente. Esta limitación se refleja claramente en los valores de AUROC y la alta tasa de falsos positivos observados durante la evaluación con OC-SVM.

4.5.4. Entrenamiento por tripletas para el reconocimiento de gestos

Con el objetivo de generar representaciones discriminativas de gestos aislados en lengua de signos española, se ha implementado un modelo basado en redes LSTM bidireccionales entrenado mediante aprendizaje por tripletas (*triplet learning*), una técnica ampliamente utilizada en tareas de reconocimiento facial, verificación biométrica y recuperación de imágenes.

En este experimento se han implementado y comparado dos variantes existentes a partir de la función de pérdida original de *triplet loss* mencionada en la subsección **Triplet Loss y aprendizaje métrico 3.3.4.**

Batch-Hard Triplet Loss

Esta variante de la función de pérdida *Triplet Loss*, se caracteriza por seleccionar en base a un determinado ancla, el positivo más alejado a este y el negativo más próximo dentro de cada lote. Esta variante fue propuesta por Hermans et al. [65], y fuerza al modelo a aprender en los casos más difíciles, acelerando la convergencia de este:

$$L_{\text{hard}} = \frac{1}{N} \sum_{i=1}^N \max \{d(a_i, p_i) - d(a_i, n_i) + \alpha, 0\} \quad (4.15)$$

Semi-Hard Triplet Loss

Al igual que la función de pérdida *Batch-Hard Triplet Loss*, *Semi-Hard Triplet Loss* comparte la misma premisa de seleccionar a un positivo que más alejado esté del ancla, pero con la condición de seleccionar muestras negativas que estén más lejos que el positivo seleccionado, pero dentro de un margen predefinido. Este enfoque se implementó como una posible solución entre aquellos negativos que están correctamente clasificados y aquellos negativos que podrían inducir ruido o inestabilidad al entrenamiento.

El criterio de selección implica que para una tripleta, donde:

$$d(a, p) < d(a, n) < d(a, p) + \alpha \quad (4.16)$$

Este tipo de tripleta promueve un aprendizaje más estable y progresivo, ya que permite mejorar la separación entre clases sin exponer al modelo a ejemplos que resultan triviales o excesivamente ambiguos.

Evaluación del entrenamiento por tripletas

Con el objetivo de analizar la capacidad de generalización de los vectores de características generados por los modelos Bi-LSTM entrenados mediante aprendizaje por tripletas, se compararon dos variantes de la función de pérdida: *Batch-Hard Triplet Loss* y *Semi-Hard Triplet Loss*. En ambos casos, el modelo fue entrenado durante múltiples épocas hasta la convergencia, y posteriormente se extrajeron los vectores de características del conjunto de prueba. Estos vectores fueron normalizados mediante norma L_2 y utilizados para entrenar un clasificador SVM con núcleo RBF, cuyos hiperparámetros se ajustaron mediante búsqueda en cuadrícula con validación cruzada.

Cuadro 4.5: Comparación de desempeño entre estrategias de Triplet Loss

Estrategia	Val. loss mínima	Accuracy SVM (%)	F1-score ponderado
Batch-Hard	0.3820	61.6	0.61
Semi-Hard	0.0475	24.4	0.24

El modelo entrenado con Batch-Hard Triplet Loss presentó una mejora constante durante el entrenamiento, alcanzando una pérdida de validación mínima de 0.3820. El clasificador SVM entrenado sobre los vectores de características de este modelo alcanzó una tasa de acierto del 61.6 %, con una media ponderada del F1-score también del 61 %. Esta estrategia mostró un desempeño sólido en la mayoría de las clases, especialmente en aquellas con mayor representación o variabilidad estructural.

Para encontrar la combinación más adecuada de hiperparámetros, se empleó el método de *grid search*, una técnica de búsqueda exhaustiva que evalúa sistemáticamente todas las combinaciones posibles dentro de un conjunto predefinido de valores para cada parámetro. Esta estrategia permite optimizar el rendimiento del modelo ajustando parámetros clave como C y γ . Los valores óptimos de los hiperparámetros encontrados mediante *grid search* fueron $C = 100$ y $\gamma = \text{scale}$, lo cual sugiere una separación eficaz entre clases en el espacio de características.

Por otro lado, el modelo con *Semi-Hard Triplet Loss* alcanzó una pérdida de validación mucho menor (0.0475), lo cual podría parecer inicialmente prometedor. Sin embargo, este descenso en la pérdida no se tradujo en una mejora efectiva del rendimiento: el clasificador SVM obtuvo una tasa de acierto del 24.4 % y un F1-score ponderado del 24 %, lo que indica una representación menos efectiva de las relaciones entre clases. Aunque algunas clases individuales mostraron una tasa de acierto elevada, el modelo fue incapaz de mantener un rendimiento consistente a lo largo del conjunto de prueba, lo que podría explicarse por una sobreoptimización local en regiones del espacio de características que no generalizan bien.

Los resultados evidencian que, a pesar de su menor pérdida numérica, el modelo entrenado con *semi-hard triplets* no logra capturar estructuras discriminativas robustas entre clases. En cambio, la estrategia *batch-hard*, al forzar al modelo a optimizar los casos más difíciles dentro de cada lote, proporciona un aprendizaje más global y generalizable, lo que la convierte en

una alternativa preferente para tareas de clasificación de gestos en entornos multiclase como es el caso de la lengua de signos.

4.5.5. Segmentación de gestos en videos

En esta sección se describe la experimentación enfocada en la detección y segmentación de gestos mediante técnicas de segmentación y detección de acciones. Para ello, se emplearon las redes convolutivas temporales, siguiendo la arquitectura propuesta por Colin Lea et al. en su trabajo 'Temporal Convolutional Networks for Action Segmentation and Detection' [55]. Este modelo ha demostrado ser eficaz en tareas de segmentación temporal de acciones complejas, superando en muchos casos a arquitecturas recurrentes como Bi-LSTM, tanto en tasa de acierto como en velocidad de entrenamiento.

Detección de gestos

En [55] se evaluaron comparativamente las variantes ED-TCN y D-TCN en varios conjuntos de datos de segmentación de acciones humanas, incluyendo el dataset otorgado por la universidad de Dundee (50 Salads)[57], MERL Shopping [66] y el dataset Georgia Tech Ego-centric Activities (GTEA)[67]. Los resultados experimentales muestran que, si bien ambas arquitecturas superan ampliamente a los modelos recurrentes en precisión y velocidad de entrenamiento, el modelo ED-TCN ofrece un mejor equilibrio entre rendimiento y simplicidad estructural.

Tal y como se mencionó en la **subsección Segmentación temporal de gestos 3.3.5**, ED-TCN se caracteriza por el uso de filtros convolucionales largos combinados con operaciones de *pooling* y *upsampling*, lo que permite capturar dependencias temporales de largo alcance con una arquitectura poco profunda. Esta propiedad la hace especialmente eficaz para evitar errores de sobresegmentación, un problema recurrente en modelos temporales, al tiempo que mantiene una alta precisión en métricas como F1 segmental y *edit distance*. En los experimentos sobre el conjunto 50 Salads, por ejemplo, la ED-TCN alcanzó valores superiores en todas las métricas respecto a la Dilated TCN, destacando especialmente en la reducción de errores de segmentación excesiva.

Además, el modelo ED-TCN requiere un número significativamente menor de capas y parámetros en comparación con el modelo D TCN, lo que reduce tanto la complejidad computacional como el tiempo de entrenamiento. Mientras que el modelo D TCN depende de múltiples bloques con convoluciones dilatadas y conexiones residuales, el modelo ED-TCN logra un campo receptivo comparable con una estructura más compacta y eficiente.

Por estas razones, en [55] se concluye que el modelo ED-TCN proporciona una solución más equilibrada para tareas de segmentación temporal, combinando precisión, robustez y eficiencia computacional. Este análisis motivó la adopción del modelo ED-TCN como arquitectura base en el presente trabajo.

Siguiendo dicha evaluación, se implementó la arquitectura ED-TCN para la segmentación y detección de gestos en vídeos de LSE. El flujo de procesamiento se diseñó a partir de segmentos temporales que representan frases completas realizadas por intérpretes. Cada segmento contiene al menos un gesto de interés (previamente filtrado) y puede incluir gestos irrelevantes, señas de lengua de signos o pausas sin gesticulación.

Para facilitar el aprendizaje de patrones temporales relevantes, se reformuló el problema como una clasificación binaria: “gesto” vs. “no gesto”. Dado que la duración entre los “gestos” y los “no gestos”, se propuso un mecanismo de *downsampling* por cajas que permite normalizar parcialmente la longitud de las secuencias.

Sea una secuencia de entrada representada por $\mathbf{X} \in R^{T \times F}$, donde:

- ✓ T es la longitud de la secuencia (número de *frames*).
- ✓ F es el número de características por *frame*.

Se busca obtener una nueva secuencia $\mathbf{X}' \in R^{L \times F}$, con $L < T$, que conserve la estructura representativa de la original.

Donde:

- ✓ L : **longitud objetivo** de la secuencia reducida. Actúa como número de intervalos (*cajas*) en que se particiona la secuencia original y determina la resolución temporal final.
- ✓ Δ : **tamaño de caja**; anchura (en frames) de cada intervalo uniforme, definida como $\Delta = T/L$.
- ✓ t_i : índice del frame representativo seleccionado en la i -ésima caja; se calcula desplazándose al centro geométrico de la caja y redondeando al entero más cercano.

Algoritmo de downsampling por cajas

El algoritmo divide la secuencia $\mathbf{X}$ en L intervalos de tamaño uniforme y selecciona el elemento central de cada uno:

Cálculo del tamaño de caja:

$$\Delta = \frac{T}{L}. \quad (4.17)$$

Para cada $i = 0, 1, \dots, L - 1$, se calcula el índice representativo

$$t_i = \min\left(T - 1, \left\lfloor \left(i + \frac{1}{2}\right) \Delta \right\rfloor\right), \quad (4.18)$$

donde $\lfloor \cdot \rfloor$ denota redondeo al entero más cercano.

Construcción de la secuencia reducida:

$$\mathbf{X}' = \begin{bmatrix} \mathbf{x}_{t_0} \\ \mathbf{x}_{t_1} \\ \vdots \\ \mathbf{x}_{t_{L-1}} \end{bmatrix} \in R^{L \times F}. \quad (4.19)$$

El resultado es una sub-muestra uniforme que mantiene la dinámica esencial de la secuencia original con complejidad $\mathcal{O}(T)$.

Algoritmo 4 Submuestreo por Cajas (Box-based Downsampling)

Require: DataFrame de entrada D con n filas, longitud objetivo $L < n$

Ensure: DataFrame reducido D' con L filas, submuestreado uniformemente

```

1: if  $n \leq L$  then
2:   return  $D$ 
3: end if
4:  $\Delta \leftarrow \frac{n}{L}$ 
5: Crear lista vacía  $I$ 
6: for  $i = 0$  to  $L - 1$  do
7:    $t_i \leftarrow \min(n - 1, \lfloor (i + \frac{1}{2}) \cdot \Delta \rfloor)$ 
8:   Añadir  $t_i$  a  $I$ 
9: end for
10:  $D' \leftarrow$  seleccionar filas de  $D$  con índices en  $I$ 
11: return  $D'$ 
```

Evaluación del modelo ED-TCN

Tras aplicar la técnica de *downsampling* por cajas al conjunto de datos de segmentos, este fue particionado en tres subconjuntos: entrenamiento, validación y prueba, aplicando *padding* a todas las secuencias, para asegurarnos una duración similar de cada muestra, y empleando una función de pérdida *focal loss* para mitigar el desbalance entre clases.

La red se entrenó con una arquitectura de dos capas temporales convolucionales, con 64 y 128 nodos respectivamente, alcanzando un *accuracy* del 90 % en el conjunto de evaluación.

Sin embargo, este rendimiento refleja un escenario ideal en el que se conoce de antemano el momento de inicio y fin de cada gesto, permitiendo extraer únicamente los segmentos relevantes para el entrenamiento y la evaluación. En un escenario más realista, donde se desea detectar gestos directamente sobre secuencias completas sin conocimiento previo de sus límites temporales, el rendimiento disminuye drásticamente. Al aplicar el modelo a estas secuencias sin recorte explícito, se observa un tasa de acierto del 32 %, lo que evidencia una fuerte dependencia del modelo respecto al tipo de datos con los que ha sido entrenado.

Esta discrepancia sugiere que, aunque el modelo ha aprendido a distinguir eficazmente entre gestos y fondo dentro de segmentos bien definidos, su capacidad generalizadora se ve

comprometida cuando se enfrenta a secuencias más largas y con menor proporción de frames positivos.

Capítulo 5

Evaluación

En este capítulo se realizará una recapitulación de los resultados obtenidos, durante las fases de desarrollo del sistema de interpretación automática de gestos de la lengua de signos española, y posteriormente se realizará una evaluación general del proyecto. El proceso se basó íntegramente en la aplicación de la metodología **CRISP-DM**, lo que permitió iterar sobre distintas fases del ciclo de vida del análisis de datos.

A partir de la clasificación de gestos aislados, se avanzó hacia el reconocimiento de gestos en videos, la detección de novedades en gestos, la mejora de representaciones mediante *Triplet Loss* y, finalmente, la segmentación temporal automática de gestos.

5.1. Clasificación de gestos aislados

La primera fase del desarrollo se centró en la clasificación de gestos aislados, como punto de partida para evaluar la capacidad de los modelos de redes neuronales recurrentes para aprender patrones temporales. Se entrenaron modelos Bi-GRU y Bi-LSTM sobre conjuntos de datos procesados mediante dos estrategias: *padding* y muestreo. Los modelos fueron optimizados con funciones de pérdida *Cross-Entropy* y *Focal Loss*, variando el parámetro de enfoque γ .

Los mejores resultados se obtuvieron con el modelo **Bi-LSTM + Focal Loss** ($\gamma = 3$) sobre el conjunto con *padding*, alcanzando una **tasa de acierto del 78.7%**, seguido muy de cerca por el mismo modelo con $\gamma = 2$ sobre el conjunto de muestreo, con una **tasa de acierto del 77.8%**. Estos resultados demuestran que la Focal Loss con un valor intermedio de γ mejora el rendimiento del modelo, especialmente en la clasificación de clases minoritarias o más difíciles.

5.2. Reconocimiento de gestos en video

Una vez alcanzados buenos resultados en condiciones controladas, se evaluó la capacidad de generalización de los modelos entrenados sobre secuencias de vídeo no segmentadas, simulando una conversación real. Para ello, se aplicó una estrategia de *ventanas deslizantes* sobre las secuencias completas, clasificando cada ventana de forma independiente.

Los resultados fueron significativamente inferiores, con una **tasa de acierto del 1 %**. Esta caída abrupta se debe a la fragmentación de los gestos dentro de las ventanas móviles, lo que dificulta su reconocimiento. La baja precisión evidenció la necesidad de incorporar mecanismos más robustos para detectar gestos completos dentro de secuencias no segmentadas.

5.3. Detección de novedades

Ante las limitaciones del modelo en escenarios abiertos, se investigó la posibilidad de incorporar detección de gestos no conocidos mediante técnicas de *novelty detection*. Se re-utilizaron los vectores de características generados por el modelo Bi-LSTM para entrenar clasificadores como el *One-Class SVM (OC-SVM)*.

Aunque el rendimiento fue moderado, los resultados muestran que el sistema es capaz de distinguir entre clases conocidas y no vistas, lo que representa un paso inicial hacia una arquitectura más flexible y extensible, con capacidad para incorporar nuevas clases en el futuro.

5.4. Representaciones por tripletas

Con el objetivo de mejorar la discriminación entre gestos en el espacio de características, se exploró el entrenamiento con *Triplet Loss*, comparando dos estrategias de selección de tripletas: *Semi-Hard* y *Batch-Hard*.

Los resultados mostraron una clara ventaja de la estrategia **Batch-Hard**, que alcanzó una **tasa de acierto del 61.6 %** al clasificar los vectores de características con SVM, frente al **24.4 %** obtenido con *Semi-Hard*. Esto sugiere que seleccionar tripletas más difíciles durante el entrenamiento mejora la calidad de la representación aprendida, facilitando la separación entre clases en tareas de clasificación o detección.

5.5. Segmentación automática de gestos

Finalmente, se abordó la segmentación temporal de gestos como una alternativa más robusta al uso de ventanas deslizantes. Se entrenó un modelo ED-TCN sobre vídeos anotados con los límites temporales de cada gesto.

El modelo alcanzó una **tasa de acierto del 90 %** en escenarios con segmentaciones manuales. No obstante, al aplicarlo a vídeos no segmentados previamente, el rendimiento se redujo al **32 %**, revelando su dependencia del conocimiento explícito sobre el inicio y el fin de cada gesto. Este resultado reafirma la necesidad de combinar segmentación y clasificación en un enfoque conjunto para abordar el reconocimiento continuo de gestos en vídeos reales.

5.6. Evaluación completa del proyecto

En este trabajo de fin de grado se ha tratado de integrar métodos de visión por computador y aprendizaje profundo para la traducción de la lengua de signos española. A lo largo del desarrollo, se ha seguido la metodología **CRISP-DM**, lo que ha permitido estructurar el proyecto en fases bien definidas, facilitando la iteración entre etapas según las necesidades experimentales.

Desde una perspectiva técnica, los resultados obtenidos en cada una de las fases muestran un progreso coherente hacia el objetivo de construir un sistema automático capaz de reconocer gestos de la LSE a partir de vídeos. El sistema fue capaz de alcanzar una tasa de acierto del al 79 % en **ISRL**, lo cual valida la capacidad de los modelos **Bi-LSTM** para capturar patrones temporales relevantes a partir de representaciones basadas en distancias corporales. Así mismo, se confirmó que el uso de *Focal Loss* con valores adecuados del parámetro de enfoque mejora la clasificación en contextos con clases desbalanceadas o difíciles de distinguir.

No obstante, la transición hacia **CSLR** reveló importantes limitaciones. El uso de ventanas deslizantes produjo una pérdida significativa de tasa de acierto, evidenciando la sensibilidad del modelo a secuencias incompletas o mal alineadas. Para mitigar este problema, se introdujo un enfoque de segmentación temporal automática mediante **ED-TCN**, que mejoró el rendimiento en secuencias con anotaciones, pero también mostró grandes pérdidas de tasa de acierto en condiciones más realistas, como vídeos sin segmentación previa.

Por otro lado, se exploró la *detección de novedades* y el aprendizaje de representaciones más discriminativas mediante *Triplet Loss*. Los resultados obtenidos con estrategias como *Batch-Hard* muestran que es posible construir espacios embebidos donde gestos de clases distintas se separan de manera más efectiva, y donde clasificadores externos, como **SVM**, pueden operar con un rendimiento aceptable. La inclusión de métodos de detección de novedad representa un paso importante hacia sistemas capaces de reconocer gestos no vistos previamente y adaptarse a nuevas clases sin reentrenamiento completo.

A nivel metodológico, el enfoque **CRISP-DM** resultó especialmente útil para guiar el proceso de desarrollo iterativo, permitiendo volver a etapas previas, como la preparación de los datos o la redefinición de objetivos en las fases de modelado, en función de los resultados observados. Este enfoque facilitó la identificación temprana de cuellos de botella, como la escasa anotación de gestos dentro de las frases, y motivó la incorporación de nuevas técnicas, como la segmentación temporal.

Finalmente, desde un punto de vista práctico, se identifican varias limitaciones que deben ser abordadas en trabajos futuros, como puede ser la necesidad de un dataset más diverso y

representativo, la mejora del preprocesamiento para reducir la variabilidad entre intérpretes, y la exploración de arquitecturas híbridas que combinen clasificación, detección y segmentación de manera conjunta y robusta.

En conjunto, los resultados alcanzados sientan una base sólida para el desarrollo de sistemas de **ISLR**. Sin embargo, para el caso de sistemas más complejos, como es el caso de **CSLR**, se requiere de una integración más profunda entre segmentación y reconocimiento, así como de mecanismos más sofisticados para manejar la variabilidad en las secuencias de entrada.

Capítulo 6

Conclusiones y trabajo futuro

En este capítulo se expondrán las conclusiones de este trabajo de fin de grado, así como las posibles experimentos o aplicaciones que se podrían implementar a modo de trabajo futuro.

6.1. Conclusiones

Este trabajo de fin de grado ha abordado el desafío de interpretar automáticamente gestos en lengua de signos española, combinando técnicas de visión por computador y aprendizaje profundo. A lo largo del desarrollo, se han cumplido los objetivos propuestos, implementando un sistema completo capaz de detectar, representar y clasificar gestos a partir de vídeos de intérpretes.

En primer lugar, se ha demostrado la viabilidad de utilizar redes neuronales recurrentes (Bi-GRU y Bi-LSTM) para la clasificación de gestos aislados, alcanzando niveles de tasa de acierto superiores al 78 % en las mejores configuraciones. La introducción de funciones de pérdida como *Focal Loss* ha permitido mejorar el reconocimiento en clases desbalanceadas, siendo esta una aportación relevante para aplicaciones prácticas donde la frecuencia de aparición de gestos puede ser desigual.

Por otro lado, los resultados obtenidos en tareas más complejas como la detección de gestos en vídeos no segmentados o la detección de gestos novedosos han evidenciado limitaciones importantes. Aunque el sistema es capaz de aprender representaciones discriminativas a partir de tripletas y mostrar cierto nivel de generalización, el rendimiento en escenarios realistas aún se encuentra lejos de lo deseable, especialmente cuando no se dispone de anotaciones manuales.

La experimentación con modelos de segmentación temporal como ED-TCN ha mostrado un gran potencial en contextos controlados (90 % de tasa de acierto), pero también ha puesto de manifiesto su dependencia del conocimiento explícito de los límites de los gestos, con un descenso acusado del rendimiento (32 %) cuando se aplicó a secuencias completas.

En conjunto, los resultados reflejan un progreso significativo en el diseño de soluciones automáticas para la interpretación de la lengua de signos, sentando una base sólida para su futura mejora y aplicación real. Además, el proyecto contribuye al avance de tecnologías inclusivas que favorecen la accesibilidad y la equidad en la comunicación.

6.2. Trabajo futuro

Existen múltiples líneas de mejora y expansión para este proyecto que podrían ser abordadas a modo de trabajo futuro:

- ✓ **Aumento de datos y diversidad de intérpretes:** Ampliar el conjunto de datos con más gestos y mayor variabilidad de intérpretes permitirían entrenar modelos más robustos y generalizables.
- ✓ **Mecanismos de atención y arquitecturas Transformer:** La integración de módulos de atención espaciotemporal puede mejorar tanto la clasificación como la segmentación al permitir que el modelo se centre dinámicamente en las partes más relevantes de la secuencia.
- ✓ **Optimización del *spotting* en vídeo continuo:** Explorar ventanas temporales adaptativas, modelos híbridos (CNN+RNN+Transformer) y filtros temporales (e.g., HMM o CRF) para reducir falsas detecciones y mejorar la sincronización con los límites reales de cada signo.
- ✓ **Implementación en tiempo real:** Adaptar el sistema a un entorno de ejecución en tiempo real, utilizando aceleración por GPU o técnicas de compresión de modelos, permitiría su integración en aplicaciones móviles o web.
- ✓ **Evaluación con usuarios reales:** Validar el sistema con personas usuarias de lengua de signos permitiría evaluar su usabilidad, impacto y posibles ajustes para que cumpla mejor con sus expectativas y necesidades.

En definitiva, aunque el sistema desarrollado constituye un primer paso prometedor, aún queda un amplio margen de mejora antes de su aplicación práctica en contextos reales. Este trabajo ha contribuido a sentar las bases técnicas y experimentales para el desarrollo de tecnologías inclusivas que puedan facilitar la comunicación de personas con discapacidad auditiva, promoviendo una sociedad más accesible y equitativa.

Bibliografía

- [1] World Federation of the Deaf. Human rights of deaf people, 2023. Accessed: 2025-06-20.
- [2] David E. Rumelhart and James L. McClelland. Learning internal representations by error propagation. In David E. Rumelhart, James L. McClelland, and the PDP Research Group, editors, *Parallel Distributed Processing: Explorations in the Microstructure of Cognition. Volume 1: Foundations*, pages 318–362. MIT Press, Cambridge, MA, 1987.
- [3] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation, 2014.
- [4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [5] Jose L. Alba-Castro, Manuel Vázquez-Enríquez, Ania Pérez-Pérez, Flora Mariño-Pérez, Manuel L Lema-Álvarez, Carmen Cabeza-Pereiro, Eduardo Rodríguez-Banga, Laura Docío-Fernández, Soledad Torres-Guijarro, Alba Caderno-Fernández, and Sol Cid-Álvarez. Lse-health-uvigo, 2023.
- [6] Google. *Guía de soluciones de MediaPipe*, 2025.
- [7] Noha Sarhan and Simone Frintrop. Unraveling a decade: A comprehensive survey on isolated sign language recognition. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pages 3210–3217, 2023.
- [8] Necati Cihan Camgoz, Simon Hadfield, Oscar Koller, and Richard Bowden. Neural sign language translation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7784–7793, 2018.
- [9] Samuel Albanie, Gül Varol, Liliane Momeni, Triantafyllos Afouras, Joon Son Chung, and Andrew Zisserman. Bsl-1k: Scaling up co-articulated sign language recognition using mouthing cues. *ECCV*, 2020.
- [10] Junfu Huang, Weiyue Zhou, Weizhe Zhang, Houqiang Li, and Wanqing Li. Video-based sign language recognition without temporal segmentation. *AAAI*, 2018.

- [11] Zhe Cao, Tomas Simon, Shih-En Wei, and Yaser Sheikh. Realtime multi-person 2d pose estimation using part affinity fields. In *CVPR*, 2017.
- [12] Dongxu Li, Cristian Rodriguez, Xin Yu, and Hongdong Li. Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. In *WACV*, 2020.
- [13] Oscar Koller. Weakly supervised learning with multi-stream cnn-lstm-hmms to discover sequential parallelism in sign language videos. *IJCV*, 127(5):492–512, 2019.
- [14] Seng-Beng Ong and Surendra Ranganath. Automatic sign language analysis: A survey and the future beyond lexical meaning. *IEEE TPAMI*, 27(6):873–891, 2005.
- [15] Hao Min, Jing Hao, Xiao Chai, and Jie Chen. Hand-shape-based recognition of isolated sign language gestures using deep learning. In *Sensors*, 2021.
- [16] Maneet Singh, Richa Singh, and Arun Ross. A comprehensive overview of biometric fusion. *Information Fusion*, 52:187–205, 2019.
- [17] Halil Gökçe, Hüseyin Gündüz, Aybars Alaybeyoglu, and Alptekin Temizel. Sign language recognition with multi-stream transformer networks. In *CVPRW*, 2022.
- [18] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. *CVPR*, 2009.
- [19] Joao Carreira and Andrew Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset. In *CVPR*, 2017.
- [20] Sarah Alyami, Hamzah Luqman, and Mohammad Hammoudeh. Reviewing 25 years of continuous sign language recognition research: Advances, challenges, and prospects. *Information Processing & Management*, 61:103774, 2024.
- [21] Nikolaos Koulierakis, Georgios Siolas, Eleni Efthimiou, Stavroula-Evita Fotinea, and Andreas Stafylopatis. Temporal action segmentation with temporal convolutional networks. *Neural Computing and Applications*, 2021.
- [22] Rui Wei, Yifan Zhao, Weiyue Zhou, and Liang Li. End-to-end sign language recognition with transformers. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2021.
- [23] Rupam Choudhury, Rakesh Talukdar, Monalisa Bhuyan, and Kishor Sarma. Continuous sign language recognition using movement epenthesis removal. In *2017 IEEE International Conference on Computer Vision Workshops (ICCVW)*, 2017.
- [24] Fang Yang, Dacheng Tao, and Jiabin Ye. Modeling movement epenthesis with hmm in continuous sign language recognition. *Pattern Recognition*, 52:202–211, 2016.
- [25] M. Boháček and M. Hruš. Implicit handling of movement epenthesis for sign language recognition. In *European Conference on Computer Vision Workshops (ECCVW)*, 2022.

- [26] Y. Zhang and X. Zhang. Weakly-supervised sign language recognition with hierarchical context. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [27] P. Athira, R. Sruthi, and T. S. Lijiya. Coarticulation modeling for continuous sign language recognition using deep learning. In *2019 International Conference on Vision Towards Emerging Trends in Communication and Networking (ViTECoN)*, 2019.
- [28] Chunli Wang, Wen Gao, and Zhaoguo Xuan. A real-time large vocabulary continuous recognition system for chinese sign language. In *Advances in Multimedia Information Processing*, pages 150–157. Springer Berlin Heidelberg, 2001.
- [29] Shigeyuki Tateno, Hongbin Liu, and Junhong Ou. Development of sign language motion recognition system for hearing-impaired people using electromyography signal. *Sensors*, 20(20), 2020.
- [30] Willow Garage Intel and Itseez. *OpenCV: Open Source Computer Vision Library*, 2000.
- [31] Google. *Guía de detección de puntos de referencia manuales para Python*, 2025.
- [32] Google. *Guía de detección de puntos de referencia de poses para Python*, 2025.
- [33] Sepp Hochreiter. Untersuchungen zu dynamischen neuronalen netzen. Master’s thesis, Technische Universität München, 1991.
- [34] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.
- [35] Marco A.F. Pimentel, David A. Clifton, Lei Clifton, and Lionel Tarassenko. A review of novelty detection. *Signal Processing*, 99:215–249, 2014.
- [36] Bernhard Schölkopf, John C. Platt, John Shawe-Taylor, Alex J. Smola, and Robert C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7):1443–1471, 2001.
- [37] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer-Verlag, New York, 1995.
- [38] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.
- [39] IBM. ¿qué son las máquinas de vectores de soporte (svm)? Accessed: 2025-06-04.
- [40] David S. Broomhead and David Lowe. Multivariable functional interpolation and adaptive networks. *Complex Syst.*, 2, 1988.
- [41] Suleiman Y. Yerima and Abul Bashar. Semi-supervised novelty detection with one class SVM for SMS spam detection. In *29th International Conference on Systems, Signals and Image Processing (IWSSIP)*, pages 169–174. IEEE, 2022.

- [42] Mohammad Saleh Sadooghi and Siamak Esmailzadeh Khadem. Improving one class support vector machine novelty detection scheme using nonlinear features. *Pattern Recognition*, 83:14–33, 2018.
- [43] Florian Schroff, Dmitry Kalenichenko, and James Philbin. Facenet: A unified embedding for face recognition and clustering. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, page 815–823. IEEE, June 2015.
- [44] Wikipedia. Triplet loss. Accessed: 2025-06-04.
- [45] Oscar Koller, Hermann Ney, and Richard Bowden. Deep hand: How to train a cnn on 1 million hand images when your data is continuous and weakly labelled. In *European Conference on Computer Vision (ECCV)*, pages 704–720. Springer, 2016.
- [46] Ruizhi Cui, Hu Liu, and Changsheng Zhang. A deep neural framework for continuous sign language recognition by iterative training. *IEEE Transactions on Multimedia*, 21(7):1880–1891, 2019.
- [47] Bing Zhou, Zhi Zhou, Zhiqiang Zhou, and Xi Li. Spatial-temporal multi-cue network for continuous sign language recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12634–12643, 2021.
- [48] Jianfeng Min, Tianyun Zhang, Haoxuan Sun, Jianfeng Wang, and Zheng-Jun Zha. Visual alignment constraint for continuous sign language recognition. *IEEE Transactions on Image Processing*, 30:6310–6325, 2021.
- [49] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *ArXiv*, abs/1803.01271, 2018.
- [50] Alex Graves, Santiago Fernandez, and Jürgen Schmidhuber. Bidirectional lstm networks for improved phoneme classification and recognition. In *International Conference on Artificial Neural Networks*, pages 799–804. Springer, 2005.
- [51] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Gated feed-back recurrent neural networks. *arXiv*, 2015.
- [52] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pages 1310–1318, 2013.
- [53] B. Singh, T. K. Marks, M. Jones, O. Tuzel, and M. Shao. A multi-stream bi-directional recurrent neural network for fine-grained action detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1961–1970, 2016.
- [54] De-An Huang, Li Fei-Fei, and Juan Carlos Niebles. Connectionist temporal modeling for weakly supervised action labeling. In *European Conference on Computer Vision*, pages 137–153. Springer, 2016.

- [55] Colin Lea, Michael D. Flynn, René Vidal, Austin Reiter, and Gregory D. Hager. Temporal convolutional networks for action segmentation and detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 156–165, 2017.
- [56] Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew W. Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. In *Speech Synthesis Workshop*, 2016.
- [57] Sebastian Stein and Stephen Mckenna. Combining embedded accelerometers with computer vision for recognizing food preparation activities. *UbiComp 2013 - Proceedings of the 2013 ACM International Joint Conference on Pervasive and Ubiquitous Computing*, pages 729–738, 09 2013.
- [58] IBM. Conceptos básicos de ayuda de crisp-dm. Accessed: 2025-06-04.
- [59] Karen Sparck Jones. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21, 1972.
- [60] Claire C. Gordon, Cynthia L. Blackwell, Bruce Bradtmiller, Joseph L. Parham, Patricia Barrientos, Steven P. Paquette, Brian D. Corner, Jeremy M. Carson, Joseph C. Venezia, Belva M. Rockwell, Michael Mucher, and Shirley Kristensen. *Anthropometric Survey of U.S. Army Personnel: Methods and Summary Statistics*. U.S. Army Natick Soldier Research, Development and Engineering Center, Natick, MA, 2014. Technical Report.
- [61] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
- [62] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for on-line learning and stochastic optimization. *Journal of Machine Learning Research*, 12(61):2121–2159, 2011.
- [63] Geoffrey Hinton. Coursera neural networks for machine learning lecture 6, 2018.
- [64] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection, 2018.
- [65] Alexander Hermans, Lucas Beyer, and B. Leibe. In defense of the triplet loss for person re-identification. *ArXiv*, abs/1703.07737, 2017.
- [66] Bharat Singh, Tim K. Marks, Michael Jones, Oncel Tuzel, and Ming Shao. A multi-stream bi-directional recurrent neural network for fine-grained action detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1961–1970, 2016.
- [67] Alireza Fathi, Xiaofeng Ren, and James M. Rehg. Learning to recognize objects in egocentric activities. In *CVPR 2011*, pages 3281–3288, 2011.